

Company OS

Employee Management, Security and CRM Platform

System Design and Development Documentation

Version 1.0

3 October 2026

Prepared by DEVSYNX Private Limited

www.devsynx.com

Contents

Company OS - Development Documentation

1. What this is
2. Document set
3. Conventions
4. Glossary

01 - Overview, Design Review and Scope

1. System summary
2. Review of the original design
3. Scope
4. Assumptions
5. Open questions for the client

02 - Architecture

1. Technology decisions
2. Logical architecture
3. Desktop architecture
4. Repository layout
5. Deployment
6. Background jobs
7. Non-functional requirements

03 - System Flows

- F1 - Setup and employee creation
- F2 - First login, device registration, consent
- F3 - Normal login, session, logout
- F4 - Password reset and lockout
- F5 - Shift lifecycle
- F6 - Activity tracking and sync
- F7 - Offline mode (tracker never stops)
- F8 - Attendance computation
- F9 - Attendance correction and leave
- F10 - CRM lead lifecycle
- F11 - Task lifecycle
- F12 - Call logging
- F13 - Targets
- F14 - Email
- F15 - Notifications
- F16 - Offboarding
- F17 - Lost, stolen or replaced device
- F18 - Desktop auto-update
- F19 - Viewing employee data (admin side)
- F20 - Daily journey summary

04 - Module Specification

1. Authentication (AUTH)
2. Devices (DEV)
3. Organisation and employees (ORG)
4. Roles and permissions (RBAC)
5. Shift and attendance (ATT)
6. Activity tracking (TRK)
7. CRM (CRM)
8. Tasks (TASK)
9. Targets (TGT)
10. Email (MAIL)

11. Notifications (NOTIF)
12. Reports (RPT)
13. Audit and security (AUD)
14. Settings (SET)
15. Desktop App screens
16. Admin Panel screens

05 - Roles and Permissions

1. Model
2. Permission catalog
3. Default role matrix
4. Rules
5. Implementation notes

06 - Data Model

1. Identity and organisation
2. Attendance
3. Tracking
4. CRM and tasks
5. Targets
6. Email
7. Platform
8. Retention defaults
9. Key relations

07 - API Specification

1. Conventions
2. Auth and devices
3. Organisation
4. Shift and attendance
5. Tracking
6. CRM, tasks, calls
7. Targets
8. Mail
9. Notifications, reports, audit, settings, releases
10. WebSocket

08 - Security and Privacy

1. Threats considered
2. Authentication details
3. Transport and data protection
4. Application security checklist
5. Desktop and agent integrity
6. Monitoring transparency and legal basis
7. Audit
8. Operations

09 - Tracker Agent, Watchdog and Browser Extension

1. Processes
2. What the agent collects and how
3. Segments
4. Local queue and pipe protocol
5. Browser extension
6. Watchdog service
7. Installer
8. Performance and quality budgets
9. Test matrix

10 - Google Workspace Email Integration

1. Principle
2. Connection method
3. One-time setup (client Super Admin + developer)
4. Flows
5. Permission mapping
6. Shared mailbox behaviour
7. Limits and error handling
8. Offboarding effect
9. Out of scope for 1.0

11 - Build Plan

1. Phases
2. Phase details
3. Definition of done (every feature)
4. Testing strategy
5. Rollout
6. Risks
7. Items needed from the client

12 - AI Build Guide

1. Working method
2. Suggested build order (slices)
3. AGENTS.md (copy into the repository root)
4. Prompt template for one slice
5. Review checklist for AI output
6. Known traps for AI-generated code in this project

Company OS - Development Documentation

Employee Management + Security + CRM Platform

Field	Value
Version	1.0
Date	2026-10-03
Prepared by	DEVSYNX Private Limited
Status	Ready for development, pending the open questions in file 01
Working title	Company OS (rename freely)

1. What this is

One platform with which a company controls identity, CRM, company email access, attendance, targets, activity tracking, devices and security from a single Admin Panel. Employees receive only a Windows desktop application and log in with company-issued credentials.

2. Document set

File	Content	Read when
00-README.md	Index, conventions, glossary	First
01-overview-and-scope.md	System summary, review of the original design with fixes, scope, assumptions, open questions	First
02-architecture.md	Components, tech stack, repo layout, deployment, non-functional requirements	Before any code
03-flows.md	Every end-to-end flow with states, rules and edge cases	Before each feature
04-modules-spec.md	Functional requirements per module with IDs, screen lists	During each feature
05-roles-permissions.md	Permission keys, scopes, default role matrix	Auth and every endpoint
06-data-model.md	Database tables, columns, indexes, retention	Backend work
07-api-spec.md	REST and WebSocket contracts	Backend and clients
08-security-privacy.md	Auth design, device binding, hardening, monitoring transparency, retention	Throughout
09-tracker-agent-spec.md	Windows tracker agent, watchdog service, browser extension	Tracker work
10-google-workspace-email.md	Gmail integration design and setup	Email module
11-build-plan.md	Phases, deliverables, acceptance criteria, testing, risks	Planning
12-ai-build-guide.md	How to drive AI coding models with these documents, rules file, prompt templates	Every AI session

3. Conventions

- Requirement IDs look like `FR-AUTH-03`. Reference them in commits, tickets and AI prompts.
- MUST = mandatory for release. SHOULD = expected unless there is a documented reason. MAY = optional.
- Flow IDs look like `F5`. Decision IDs look like `D3`. Open questions look like `Q4`.
- All timestamps are stored in UTC. Display uses the company timezone unless stated.
- "Server" always means the central backend. "App" means the desktop application. "Agent" means the tracker agent process.

4. Glossary

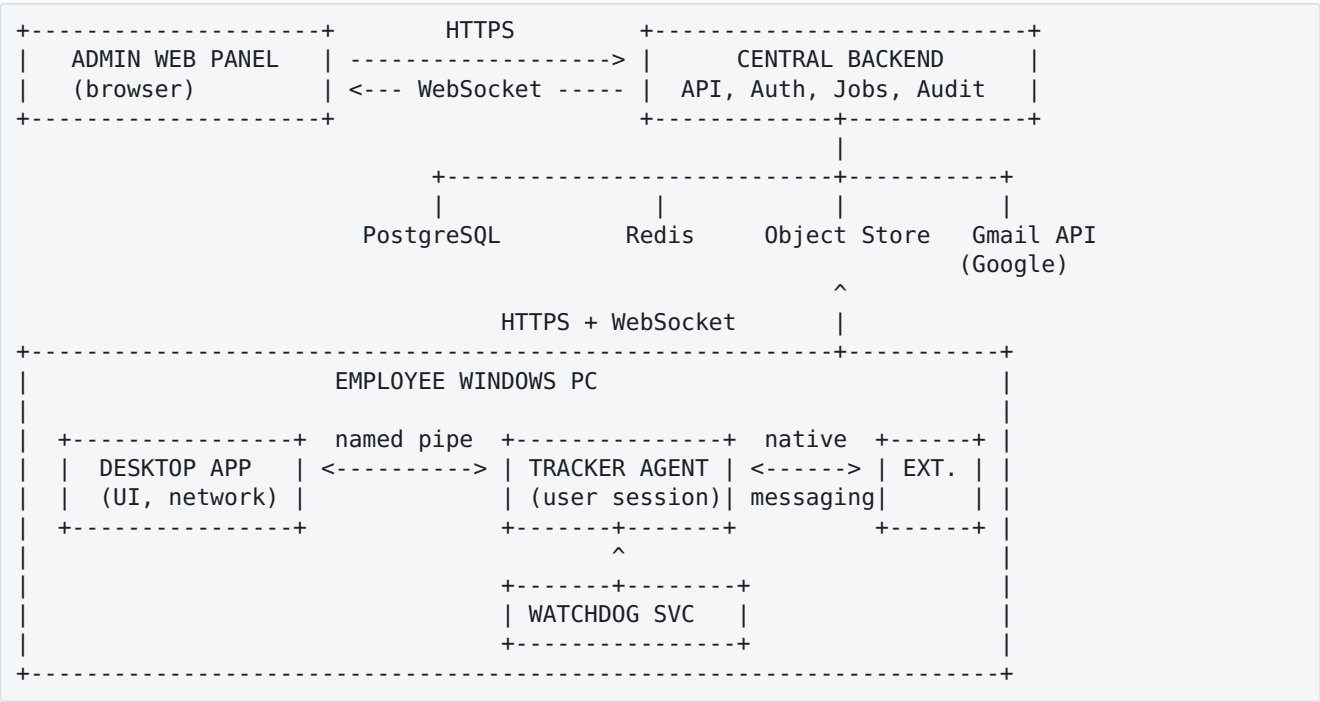
Term	Meaning
Admin Panel	Web application for Super Admin, Admin, Manager and Team Leader
Desktop App	Windows application used by employees (UI)
Tracker Agent	Background process in the employee's Windows session that measures activity
Watchdog Service	Windows service that keeps the agent alive and applies updates
Shift	One continuous working session of an employee, from Start Shift to End Shift
Segment	One continuous period of a single activity (one app or one website) inside a shift
Active time	Shift time with keyboard or mouse input, or covered by an activity exception (call, meeting)
Idle time	Shift time without input beyond the idle threshold
Rollup	Pre-aggregated daily totals per employee, app and domain
Mailbox	A Google Workspace mailbox connected to the platform (for example sales@company.com)
Scope (permission)	The data range a permission applies to: own, team, department, all
Attendance date	The calendar date a shift belongs to, defined by the scheduled shift start

01 - Overview, Design Review and Scope

1. System summary

Company OS has five deliverable components.

#	Component	Users	Purpose
1	Admin Web Panel	Super Admin, Admin, Manager, Team Leader	Manage employees, devices, CRM, targets, mailboxes, attendance, reports, security
2	Central Backend (API)	All clients	Authentication, business logic, data, Gmail integration, jobs, audit
3	Employee Desktop App (Windows)	Employee	Dashboard, CRM, tasks, email, targets, attendance, shift control
4	Tracker Agent + Watchdog Service (Windows)	None (background)	Measures apps, websites, active and idle time during a shift
5	Browser Extension (Chrome, Edge)	None (background)	Reports the active website domain to the agent



2. Review of the original design

The original concept is sound: one platform, employees never receive Google passwords, the admin controls everything. The items below were wrong, undefined or missing. Each one is fixed in this document set.

2.1 Corrections (things that would not work as written)

#	Problem in original	Why it fails	Fix	Where
C1	Tracker built as a "Native Windows	Windows services run in Session 0 and cannot see the user's foreground window or keyboard	Tracking runs in a user-session agent process. A small service acts only as watchdog and updater	09

	Service" only	and mouse input		
C2	"Browser Websites" tracking with no mechanism	A native process cannot reliably read the URL of the active tab	Force-installed browser extension reports the active domain to the agent through native messaging. Fallback is app-level time only	09
C3	Gmail through "OAuth / Google APIs" with per-permission toggles	Gmail API scopes are coarse. "Read yes, delete no" cannot be granted by Google per employee	Backend is the only Gmail client. Permissions are enforced by the backend, endpoint by endpoint. Connection method defined (per-mailbox OAuth, internal app)	10
C4	Targets for "Calls" and "Connected Calls"	No data source for calls exists in the design	Call logging module in CRM. All target metrics are computed from recorded events, never typed in by hand	03 F12, F13
C5	Idle time = no keyboard or mouse	Employees on phone calls or meetings produce no input and would be recorded as idle	Activity exceptions: listed call and meeting apps in foreground, or microphone in use, count as active	09
C6	Attendance shows Late and Absent	No shift schedule exists to compare against	Shift schedules, grace period, holidays, leave, weekly off days	03 F8
C7	Fixed 5-level role ladder	Real permission needs do not follow a strict ladder (for example a Team Leader who may reassign leads but not see tracking)	Permission keys with data scopes. The five roles are default bundles and remain editable	05
C8	Three languages (Flutter, C#, Python or Node)	Larger surface, no shared types, harder for AI-assisted development	TypeScript everywhere (backend, admin, desktop UI) plus C# only for the agent	02

2.2 Gaps (things not addressed)

#	Gap	Fix	Where
G1	Device authorization had no process	Device registration with admin approval, device key pair, revoke	03 F2, F17
G2	No offline behaviour	Offline-first tracker: keeps working with no internet, local encrypted queue, automatic sync and backfill in the Admin Panel on reconnect	03 F7
G3	No handling for forgotten End Shift, crash, sleep, lock, clock change	Shift state machine with heartbeat and auto-close rules	03 F5
G4	Time taken from the employee PC	Server time is authoritative. Durations use a monotonic clock	03 F5, 08
G5	Offboarding did not cover owned data	Mandatory reassignment of leads, tasks and follow-ups before disable completes	03 F16
G6	Shared mailbox with no attribution	Email audit log: which employee read, sent or replied from which mailbox	10
G7	Raw activity events at unlimited volume	Agent uploads segments, server keeps daily rollups, raw data has a retention period	06, 09
G8	No transparency or consent for monitoring	Consent screen, visible tracking indicator, tracking only during a shift, monitoring policy text	08
G9	No installer signing or auto-update	Signed installer and binaries, staged auto-update	03 F18, 09
G10	Admin Panel security	Mandatory TOTP 2FA for all admin panel roles, IP allowlist	08

	unspecified	option, session controls	
G11	Employees could bulk-export leads	Export is a separate permission, off for employees, all exports audited	05
G12	"Creates Company" implies multi-tenant, undefined	Single deployment for this client, schema is tenant-ready (company_id on all rows)	02 D7
G13	No lead import or duplicate handling	CSV import with duplicate detection on normalized phone and email	04
G14	No productivity classification	App and website categories (productive, neutral, unproductive) per department	04
G15	No attendance correction or leave	Correction requests with approval, basic leave requests	03 F9
G16	No backup, monitoring, environments	Defined in architecture and security	02, 08
G17	Password reset undefined	Admin-initiated reset with forced change, lockout policy	03 F4
G18	Night shifts crossing midnight	Attendance date bound to scheduled shift start, not calendar midnight	03 F8

3. Scope

3.1 In scope, version 1.0

1. Authentication: employee ID or company email + password, device registration and approval, sessions, lockout, admin 2FA.
2. Organisation: company, departments, teams, employees, roles, permissions.
3. Shift and attendance: start, break, resume, end, schedules, holidays, late and absent rules, corrections, basic leave.
4. Activity tracking: active and idle time, applications, website domains, keyboard and mouse counts (counts only), lock and unlock, offline gaps.
5. CRM: leads, pipeline stages, notes, call logs, appointments, follow-ups, attachments, timeline, CSV import, duplicate detection, reassignment.
6. Tasks: assign, priority, deadline, status, link to lead.
7. Targets: daily, weekly, monthly, per employee or team, automatic progress.
8. Email: connect Google Workspace mailboxes, assign to employees with permissions, read, send, reply, draft, attachments, audit log.
9. Notifications: in-app real time and Windows toast.
10. Reports: attendance, time, applications, websites, productivity, CRM, targets, with CSV export.
11. Security: audit log, login history, device and session management, RBAC.
12. Desktop delivery: signed installer, auto-update.

3.2 Version 1.1 (after pilot)

- Optional screenshots (off by default, see 08).
- Scheduled email reports to managers.
- Google sign-in (SSO) as an alternative login for employees who own a Workspace account.
- Firefox extension.
- PDF export of reports.

3.3 Out of scope

- Keystroke content recording, clipboard capture, webcam, screen recording video. Not built.
- Payroll and salary calculation.
- Built-in telephony or dialer. Calls are logged, not placed, by the system (see Q2).
- macOS and Linux desktop app (see Q6).
- Mobile apps.
- Google Calendar, Drive and Chat integration.
- Website blocking. The system measures, it does not block.

4. Assumptions

#	Assumption
A1	One client company, up to about 200 employees, Windows 10 and 11 PCs
A2	The client owns a Google Workspace domain and the client's Super Admin will perform the one-time Google-side setup steps
A3	Employees do not have local administrator rights on their PCs (strongly recommended, otherwise tamper resistance is limited)
A4	Employees work fixed or rotating shifts that can be described as schedules, including night shifts
A5	Employees are informed about monitoring in writing before rollout (client responsibility, see 08)
A6	Chrome or Edge is the standard browser on employee PCs
A7	The client provides or pays for a code signing certificate and hosting

5. Open questions for the client

Development can start without these answers. The default in the last column is built unless the client decides otherwise.

#	Question	Why it matters	Default if unanswered
Q1	Does the built-in CRM replace the tools seen in the example (GoHighLevel, Lofty) or run next to them?	If they stay, leads and calls live elsewhere and targets need an integration instead of manual logging	Built-in CRM is the system of record. No integration in 1.0
Q2	Which phone system or dialer do employees use for calls?	Automatic call counts need an integration with that system	Manual call logging in the CRM with outcome and duration
Q3	How many mailboxes and are they shared (sales@) or personal (name@)?	Decides mailbox connection method and assignment UI	Fewer than 20, mostly shared, per-mailbox OAuth
Q4	Where are employees located and which law applies to monitoring?	Consent and retention requirements differ by country	Consent screen, written policy, 90 days raw retention
Q5	Are screenshots required?	Storage cost, privacy impact	Not in 1.0
Q6	Any macOS users?	Separate agent implementation	Windows only
Q7	Shift patterns: fixed hours, rotating, night shifts, weekly off days?	Attendance rules	Configurable schedules, one default schedule Monday to Friday
Q8	Are breaks paid, and is there a daily break allowance?	Working time calculation	Breaks are unpaid and excluded from working time, allowance 60 minutes, excess flagged
Q9	Who approves new devices: any	Security flow	Any role holding the

	Admin or only Super Admin?		<code>devices.approve</code> permission
Q10	Should employees see their own tracked data (apps and websites)?	Transparency, trust	Yes, own data visible in the app
Q11	Hosting preference and data location (region)?	Compliance, latency	Single cloud VPS setup in a region near the employees
Q12	Will this be resold to other companies later?	True multi-tenancy, billing, Google app verification	No. Tenant-ready schema only

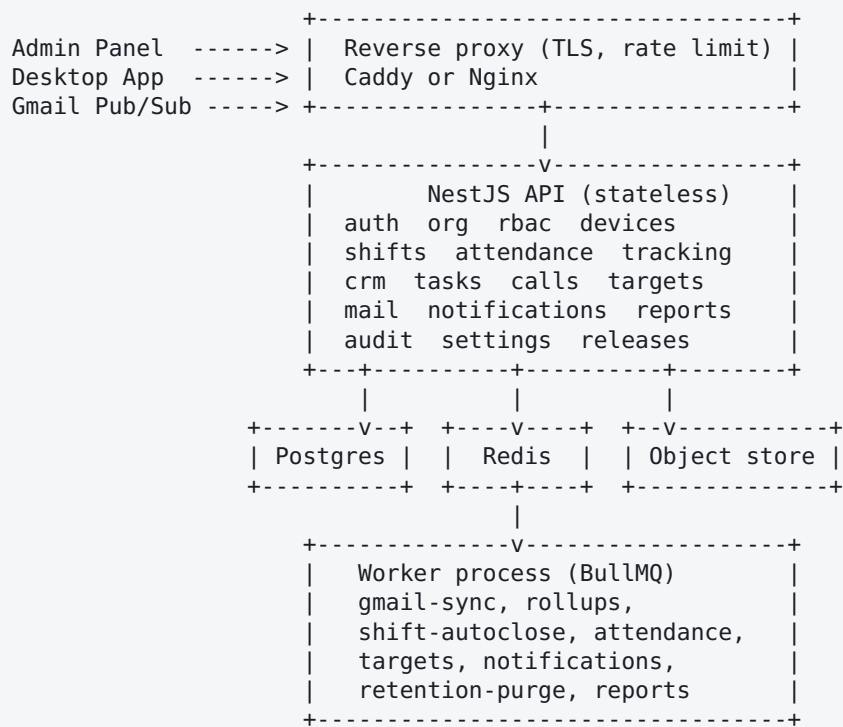
02 - Architecture

1. Technology decisions

ID	Decision	Choice	Reason
D1	Backend	Node.js 22 LTS, NestJS, TypeScript	Structured modules, guards for RBAC, good fit for AI-assisted work, shared types with clients
D2	Database	PostgreSQL 16, Prisma ORM, raw SQL for partitioned activity tables	Relational data, partitioning, reliable
D3	Cache, queues, pub/sub	Redis 7, BullMQ	Jobs (Gmail sync, rollups, auto-close), WebSocket fan-out, rate limits
D4	Admin Panel	Next.js (App Router), React, TypeScript, Tailwind, shadcn/ui, TanStack Query	Standard, fast to build
D5	Desktop App UI	Electron + React + TypeScript (electron-vite, electron-builder)	Same language and UI components as the Admin Panel, native HTML email rendering, mature auto-update
D6	Tracker Agent and Watchdog	C# on .NET 8, self-contained build	Direct Win32 access, small footprint, no runtime install
D7	Tenancy	Single deployment, every table carries <code>company_id</code>	Ready for more companies later without a rewrite
D8	Realtime	WebSocket (Socket.IO gateway in NestJS, Redis adapter)	Notifications, force logout, live dashboards
D9	File storage	S3-compatible object storage, private bucket, presigned URLs	Attachments, optional screenshots, installers
D10	Validation and contracts	Zod schemas in a shared package, OpenAPI generated from the backend	One source of truth for request and response shapes

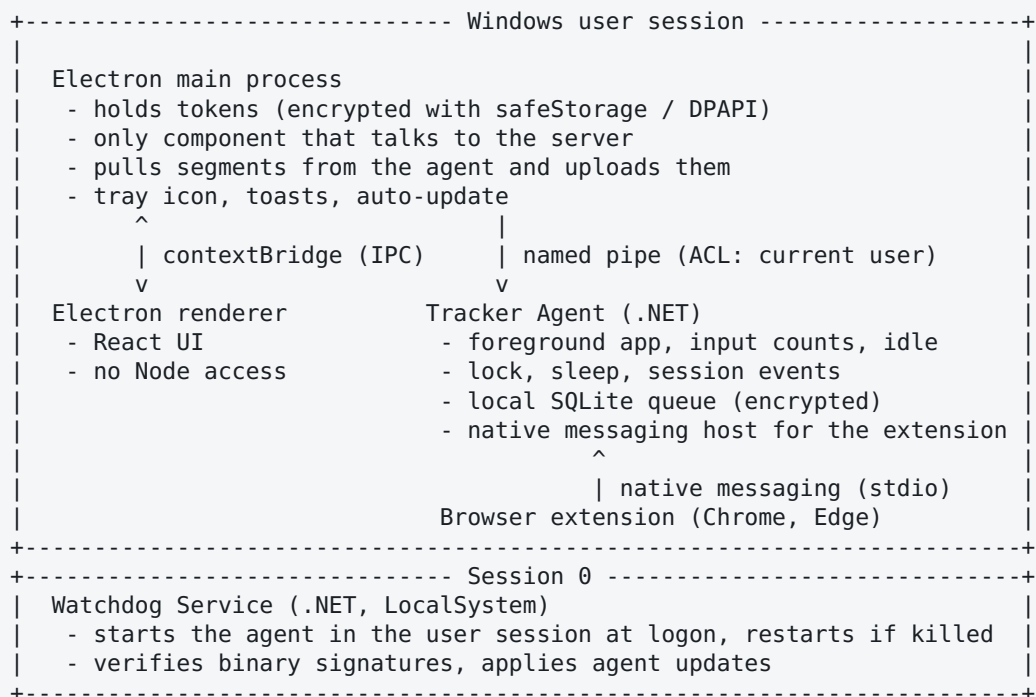
Note on Flutter: the original proposal used Flutter for the desktop UI. Flutter desktop is workable, but it adds a third language, cannot share components with the Admin Panel and needs an embedded WebView to render HTML email. If Flutter is still preferred, only `apps/desktop` changes. The backend, the agent and all contracts stay as specified.

2. Logical architecture



The API and the worker are the same codebase started in two modes. Both are stateless and can be scaled horizontally.

3. Desktop architecture



Rules:

1. Only the Electron main process holds credentials and calls the server. The agent has no network access of its own.
2. The agent tracks only while a shift is in state WORKING. It is told to start and stop by the app over the pipe.

3. If the app crashes during a shift, the agent keeps tracking into its local queue. The watchdog relaunches the app in the tray and the shift resumes (F5). The agent stops on its own only when the maximum shift length is reached.
4. Closing the app window minimises to tray while a shift is open. Quit while on shift asks: End shift, or keep running.

4. Repository layout

One monorepo (pnpm workspaces + Turborepo) plus the .NET solution.

```
company-os/
  apps/
    api/           NestJS backend and worker
    admin/         Next.js Admin Panel
    desktop/       Electron app (main, preload, renderer)
  packages/
    contracts/     Zod schemas, DTO types, permission keys, enums, WS events
    ui/            Shared React components
    config/        eslint, tsconfig, tailwind presets
  agent/
    CompanyOS.Agent/      user-session tracker
    CompanyOS.Watchdog/   Windows service
    CompanyOS.Shared/     pipe protocol, models
    CompanyOS.Agent.Tests/
  extension/           Browser extension (Manifest V3)
  installer/           electron-builder config, NSIS scripts, policy templates
  infra/               docker-compose, Caddyfile, backup scripts
  docs/                this documentation
  AGENTS.md            rules for AI coding models (see 12)
```

5. Deployment

Item	Specification
Environments	local (docker compose), staging, production
Runtime	Docker containers: proxy, api, worker, postgres, redis. Object storage external (AWS S3, Cloudflare R2 or MinIO)
Minimum production size	4 vCPU, 8 GB RAM, 160 GB SSD for up to 200 employees. Database on a managed service is preferred
TLS	TLS 1.2+ only, HSTS, certificates by Let's Encrypt or cloud provider
Domains	api.<domain>, admin.<domain>, updates.<domain>
CI/CD	GitHub Actions: lint, typecheck, unit tests, build, migrations check, deploy to staging on main, manual promotion to production
Migrations	Prisma migrations, forward only, run before the new API version starts
Secrets	Environment variables injected from a secret manager. Never in the repository
Backups	Postgres: continuous WAL archiving or daily full backup plus point-in-time recovery, 30 days. Restore test monthly
Monitoring	Health endpoints, structured JSON logs, error tracking (Sentry), uptime check, queue depth alert, disk alert

6. Background jobs

Job	Trigger	Function

shift-heartbeat-check	Every minute	Marks offline gaps, auto-closes stale shifts (F5)
activity-rollup	On each ingest batch + nightly reconcile	Updates daily rollups per employee, app, domain
attendance-close	Per schedule, after shift end + buffer	Computes attendance status per employee and date (F8)
target-progress	On each metric event + period end	Updates progress, writes period snapshot (F13)
gmail-sync	Pub/Sub push + fallback every 2 minutes	Incremental mailbox sync (F14)
gmail-watch-renew	Daily	Renews Gmail watch for every connected mailbox
followup-reminder	Every minute	Sends due follow-up and task reminders
retention-purge	Nightly	Deletes data past its retention period (O8)
partition-maintenance	Monthly	Creates next activity partitions, drops expired ones

All jobs are idempotent and safe to retry.

7. Non-functional requirements

ID	Requirement
NFR-01	API p95 response under 300 ms for standard reads, under 1 s for reports on rollups
NFR-02	Activity ingest handles 200 concurrent agents sending a batch per minute without backlog
NFR-03	Agent CPU under 1 percent average, memory under 80 MB, no visible input lag
NFR-04	Desktop App cold start under 5 s on a standard office PC
NFR-05	Tracking never stops when the internet is down. No activity data loss for offline periods up to 30 days. Automatic backfill on reconnect
NFR-06	Availability target 99.5 percent monthly. The app keeps working offline for shift and tracking
NFR-07	All times stored as <code>timestampz</code> in UTC. Company timezone set in settings. Week start configurable
NFR-08	Server time is authoritative. Client clock offset is measured at login and on every heartbeat
NFR-09	Every list endpoint is paginated. No unbounded queries
NFR-10	Admin Panel works on current Chrome, Edge, Firefox. Desktop App supports Windows 10 21H2+ and Windows 11, x64
NFR-11	UI language English. Strings kept in resource files for later translation

03 - System Flows

Every flow lists the actor, the steps, the rules and the edge cases. Defaults in brackets are configurable in Settings.

F1 - Setup and employee creation

Actor: Super Admin, Admin.

```
Seed script creates Company + first Super Admin
-> Super Admin logs in to Admin Panel, enrolls TOTP 2FA (mandatory)
-> Sets company settings (timezone, week start, idle threshold, break rules)
-> Creates departments and teams
-> Creates shift schedules and holidays
-> Reviews default roles and permissions
-> Creates employees
-> Connects mailboxes (F14) and assigns them
-> Creates pipeline stages, target metrics, targets
```

Create employee:

1. Admin enters name, company email (optional), department, team, role, schedule, manager.
2. Server generates employee code `EMP-0001` (sequential per company, never reused).
3. Server generates a temporary password, shown once to the admin. Flag `must_change_password = true`.
4. Employee status = `invited`. Becomes `active` after first successful login.

Rules:

- Login identifier is the employee code or the company email. Both are unique per company.
- An admin can never view an existing password. Only reset it.

F2 - First login, device registration, consent

Actor: Employee.

```
Install Desktop App (signed installer, per machine)
-> Open app, enter employee code or email + temporary password
-> App generates device key pair (private key stays on the PC)
-> POST /auth/login with credentials + device info + device public key
-> Server checks credentials, account status, logout
-> Device unknown:
    create device (status = pending), notify approvers
    app shows "Waiting for device approval" and polls
-> Approver approves device in Admin Panel
-> App receives approval, completes login, receives tokens
-> Forced password change
-> Monitoring consent screen (policy text, version) -> Accept
-> Dashboard
```

Rules:

1. Device identity = server-issued `device_id` + public key. Device info (machine name, OS

version, hardware hash) is stored for display only and is not trusted as proof.

2. Every token refresh is signed with the device private key. A copied token is useless on another PC.
3. Setting `device_approval_mode`: `manual` [default], `auto_first_device` (first device of an employee is approved automatically), `off`.
4. Maximum approved devices per employee [1]. Approving a new one requires revoking the old one or raising the limit.
5. Declining consent logs the employee out. Tracking never runs without a consent record for the current policy version.
6. Pending devices expire after [7] days.

F3 - Normal login, session, logout

```
Open app -> valid refresh token on this device?
yes -> refresh silently -> Dashboard
no -> login form -> POST /auth/login (signed by device key)
    -> checks: credentials, status active, device approved,
        not locked out, consent current
    -> access token (15 min) + refresh token (30 days, rotating)
    -> open WebSocket -> Dashboard
```

Rules:

- Login does not start a shift. The employee presses Start Shift (F5). Setting `auto_start_shift_on_login` exists [off].
- Refresh tokens rotate on every use. Reuse of an old refresh token revokes the whole session family and raises a security alert.
- Logout while a shift is open is blocked: the app asks to end the shift first.
- Server can push `session.revoked` over WebSocket. The app then stops the agent, wipes tokens and returns to the login screen. If the app is offline, the next API call fails with 401 and has the same effect.
- Admin Panel login: email + password + TOTP. Session cookie, HttpOnly, Secure, SameSite=Strict, idle timeout [30 min], absolute timeout [12 h].

F4 - Password reset and logout

- [5] failed attempts within 15 minutes lock the account for [15] minutes. Counter per account and per IP.
- Employee has forgotten the password: asks an admin. Admin presses Reset Password, receives a new temporary password, `must_change_password = true`, all sessions revoked.
- Employee changes own password in Profile: current password required, all other sessions revoked.
- Password policy: minimum 10 characters, checked against a breached-password list, no forced periodic rotation.
- Admin Panel users reset their own password by email link (valid 30 minutes, single use). 2FA reset only by a Super Admin.

F5 - Shift lifecycle

States

State	Meaning	Agent
OFF_SHIFT	No open shift	Stopped
WORKING	Shift open, employee working	Tracking
ON_BREAK	Shift open, break running	Paused
ENDED	Shift closed (terminal for that shift record)	Stopped

Transitions

From	Event	To	Server action
OFF_SHIFT	Start Shift	WORKING	Create shift, set start time (server time), attendance check-in
WORKING	Start Break (type)	ON_BREAK	Create break record
ON_BREAK	End Break	WORKING	Close break record
WORKING, ON_BREAK	End Shift	ENDED	Close open break, set end time, compute totals
WORKING, ON_BREAK	Auto-close (job)	ENDED	End time = last heartbeat, flag <code>auto_closed</code> , notify employee and manager

Rules

1. One open shift per employee. Start Shift while another shift is open returns 409 with the open shift. If that shift belongs to this device it is resumed, otherwise the employee is told on which device it runs.
2. Heartbeat: the app sends a heartbeat every 60 s while a shift is open. It carries state, client clock and queue size.
3. Offline gap: no heartbeat for more than [5] minutes shows the employee as Offline on the live board. This is a display state only: tracking continues locally on the PC. When the queued segments are uploaded (F7), the period is filled with the real data.
4. Auto-close: no heartbeat for [2] hours, or shift longer than [14] hours, closes the shift provisionally. End time is the last heartbeat or the last uploaded segment, whichever is later. If the PC was only offline and data arrives later, the shift is rebuilt from that data (F7).
5. Idle: no input for [5] minutes starts an idle segment, backdated to the last input. Idle time stays inside the shift and is reported separately. It is not a break.
6. Long idle: an idle period longer than [15] minutes shows a dialog on return: "You were away for 23 minutes. Mark as: Break / Working away from PC (meeting, call)". The answer is stored as a claim on the segment. Managers see claimed time separately. Setting `long_idle_prompt` [on].
7. Screen lock: treated as idle from the moment of lock.
8. Sleep, hibernate, shutdown: agent closes the current segment. If the PC wakes within the auto-close window the shift continues and the sleep period is recorded as `offline`.
9. App crash or kill: the agent keeps tracking into its local queue, the watchdog relaunches the app in the tray, the app shows "Shift resumed" and continues. Nothing is lost.
10. Break allowance: total break time above [60] minutes per shift is flagged `break_exceeded` on the attendance day.
11. Clock: shift start, break and end timestamps are assigned by the server on receipt. When an event was performed offline, the client sends its monotonic elapsed time since the last server-confirmed sync and the server reconstructs the timestamp. Events with a reconstructed timestamp are flagged.

12. Changing the Windows clock has no effect on durations (monotonic clock). A detected jump of more than 2 minutes is logged as `clock_changed`.

Working time formula

```
shift_duration    = end - start
break_time        = sum(breaks)
offline_time      = sum(offline gaps without data)
tracked_time      = shift_duration - break_time - offline_time
active_time       = sum(active segments)
idle_time         = tracked_time - active_time
working_time      = tracked_time                (attendance and payroll view)
productive_time    = active_time in apps/sites classified productive
```

F6 - Activity tracking and sync

```
Agent (every input / window change)
-> builds segments in memory
-> closes a segment when: app changes, domain changes, active<->idle flips,
    lock, pause, or segment reaches 5 minutes
-> writes closed segments to local SQLite queue

App main process (every 60 s, and at break/end)
-> reads up to 500 queued segments from the agent over the pipe
-> POST /tracking/segments (batch, each segment has a client UUID)
-> server validates: shift belongs to employee and device, times inside
    shift window, no overlap beyond tolerance
-> server upserts by UUID (idempotent), updates daily rollups
-> responds with accepted and rejected UUIDs
-> app tells the agent to delete accepted segments
```

Segment content: see 09. In short: shift id, start, end, kind (active, idle, locked), process name, app display name, domain (browsers only), key count, mouse count, exception reason.

Rules:

- No keystroke content, no clipboard, no window titles by default (setting `capture_window_titles` [off]).
- Website tracking stores the registrable domain only (for example `youtube.com`), never the full URL. Setting `capture_page_titles` [off].
- Incognito windows are not visible to the extension. Time is recorded as browser app time with domain `unknown`. Recommend disabling incognito by Chrome policy.
- Private or sensitive domains can be listed in `tracking_exclusions`. They are stored as domain `excluded`.

F7 - Offline mode (tracker never stops)

Principle: losing the internet connection MUST NOT stop, pause or degrade tracking. The agent has no dependency on the network. It records to the local encrypted queue exactly as when online, and the Admin Panel is brought up to date automatically once the connection returns.

Internet drops during a shift

- > agent keeps tracking (it never knows about the network)
- > app upload fails -> app switches to OFFLINE mode, shows a small "Offline - tracking continues, will sync" badge
- > shift timer, break and End Shift keep working locally
- > Admin Panel live board shows the employee as "Offline (last seen 10:42)" - not as stopped or absent

Internet returns

- > app detects connectivity (retry with backoff: 5 s, 15 s, 30 s, 60 s)
- > refreshes token -> sends queued shift events in order
- > uploads queued segments in batches of 500 until the queue is empty
- > server backfills shift, rollups, attendance, targets (active_hours)
- > Admin Panel shows the complete timeline for the offline period, marked "synced late" with the sync time

Function	Offline behaviour
Tracking during an open shift	Continues without limit in time. Queue capacity [30] days or [500] MB, oldest data is never dropped silently: at 90 percent the employee and, after sync, the admin are warned
PC restart while offline	Watchdog starts the agent, app starts with Windows, open shift resumes from local state, tracking continues
Start Shift	Allowed offline when the device holds a valid session (refresh token not expired). Queued, flagged <code>offline_start</code>
Break, Resume, End Shift	Allowed, queued, flagged
Auto-close by server (F5 rule 4)	Provisional only. When late data arrives, the shift is rebuilt from the real data and the flag becomes <code>recovered</code>
CRM, tasks, email	Read-only from last cached lists. No writes offline in 1.0
First login on a device, or login after logout	Not possible offline (credentials are verified by the server)
Session revoked by admin while offline	Takes effect at the first contact with the server. Data recorded until then is still uploaded and kept

Rules:

1. Timestamps recorded offline use the agent's monotonic clock anchored to the last server-confirmed time, so a wrong or changed Windows clock does not corrupt the data.
2. Sync order on reconnect: shift events, then segments (oldest first), then heartbeat. Uploads are idempotent (client UUID per event and segment), so an interrupted sync can be repeated safely.
3. Offline time with uploaded segments counts as normal tracked time. Only periods with no data at all (PC off, agent killed) remain `offline` gaps.
4. The server accepts late segments up to [30] days old. Attendance and reports for those days are recomputed automatically.
5. A flaky connection (frequent short drops) causes no visible change for the employee. The offline badge appears only after [60] s without a successful request.

F8 - Attendance computation

Inputs: schedule assignment, holidays, approved leave, shifts.

Attendance date: a shift belongs to the date of the scheduled shift whose start is closest to the actual start (window: 4 hours before to 8 hours after the scheduled start). Night shift 20:00 to 04:00 belongs entirely to the start date. A shift with no matching schedule belongs to its start date in the company timezone and is flagged `unscheduled`.

Status per employee and date, computed by the attendance-close job [2] hours after the scheduled end and recomputed when a correction is approved:

Status	Rule
Holiday	Date is a company holiday and no shift worked
Weekly off	Not a scheduled workday and no shift worked
On leave	Approved leave covers the date
Absent	Scheduled workday, no shift, no leave
Half day	Working time under [50] percent of scheduled hours
Late	First shift start later than scheduled start + grace [10 min]
Present	Otherwise

Additional flags on the day: `left_early`, `break_exceeded`, `auto_closed`, `offline_time`, `unscheduled`, `corrected`. Live status in the admin list during the day: Working, On break, Idle, Offline, Not started.

Multiple shifts on one attendance date are allowed (split shift). Times are summed. Check-in is the first start, check-out the last end.

F9 - Attendance correction and leave

Correction:

```
Employee: Attendance -> day -> Request correction
  (type: missed start, missed end, wrong break, system issue) + times + reason
-> Manager notified -> Approve / Reject with comment
-> Approved: a manual adjustment record is added (original data untouched)
-> Attendance recomputed, day flagged "corrected", audit log entry
```

Leave (basic): employee requests dates + type (annual, sick, unpaid, other) + reason, manager approves or rejects, approved days become On leave. No balance accounting in 1.0.

Rule: tracked activity data is never edited. Corrections only add adjustment records with an approver and reason.

F10 - CRM lead lifecycle

```
Create lead (manual, CSV import)      Actor: permission crm.leads.create
-> duplicate check on normalized phone (E.164) and lowercased email
    match -> show existing lead, option: open / create anyway (permission)
-> assign owner (employee) -> owner notified
-> owner works the lead:
    log call (F12), add note, send email (F14), create task,
    schedule appointment, set follow-up date
-> owner moves stage
-> Won -> lead becomes Client (flag + won date + value)
Lost -> lost reason required
```

Default pipeline (stages are configurable per pipeline):

Order	Stage	Type
1	New Lead	open

2	Attempted	open
3	Contacted	open
4	Interested	open
5	Follow-up	open
6	Meeting Scheduled	open
7	Proposal Sent	open
8	Negotiation	open
9	Won	won
10	Lost	lost

Rules:

1. Any stage can move to any other stage. Every move is written to the timeline with actor, from, to and time. Reopening a Won or Lost lead needs permission `crm.leads.reopen`.
2. Stage type drives metrics: entering a stage of type `won` counts a closing. Stages can also be marked `counts_as_qualified` and `counts_as_meeting` for targets.
3. Every lead has exactly one owner. Reassignment (single or bulk) writes a timeline entry and notifies both employees.
4. Follow-up date due: reminder at the due time and listed under "Follow-ups Today" on the dashboard. Overdue follow-ups are highlighted and reported to the manager daily.
5. Employees see only leads in their scope (05). Export needs `crm.leads.export`.
6. Timeline is append-only: stage changes, notes, calls, emails, tasks, appointments, assignments, attachments.
7. CSV import: upload, column mapping, validation preview, duplicate report, assign rule (single owner or round robin across selected employees), import as a background job with a result report.

F11 - Task lifecycle

```

Creator (manager or employee for self) creates task:
  title, description, assignee, priority (low/normal/high/urgent),
  due date-time, optional lead link
-> assignee notified
-> statuses: To do -> In progress -> Done          (also: Cancelled)
-> reminder [30 min] before due, overdue notification to assignee and creator

```

Rules: only the assignee or a user with `tasks.manage` in scope can change status. Completing a task linked to a lead writes a timeline entry. Recurring tasks are out of scope for 1.0.

F12 - Call logging

Calls are logged by the employee from the lead screen.

```

Lead -> "Log call" (or click phone number -> tel: link opens dialer,
  app shows the Log call form with a running timer)
-> fields: direction (outbound/inbound), outcome, duration, notes,
  optional next follow-up date, optional stage change
-> saved as call_log + timeline entry + metric events

```

Outcomes: Connected, No answer, Busy, Voicemail, Wrong number, Do not call. Outcome `Connected` counts toward the Connected Calls metric. Outcome `Do not call` sets the lead flag

`do_not_call`, which blocks further call logging and shows a warning.

Rule: a call log can be edited by its author for [15] minutes, afterwards only with `crm.calls.manage`. If a telephony integration is added later (Q2), it writes the same `call_log` records with `source = integration`.

F13 - Targets

Metric catalog (system-defined, computed from events):

Metric key	Counted when
<code>calls_total</code>	<code>call_log</code> created
<code>calls_connected</code>	<code>call_log</code> created with outcome <code>Connected</code>
<code>leads_created</code>	lead created by or for the employee
<code>leads_qualified</code>	lead enters a stage marked <code>counts_as_qualified</code> (once per lead)
<code>meetings_scheduled</code>	appointment created, or lead enters a stage marked <code>counts_as_meeting</code>
<code>closings</code>	lead enters a stage of type <code>won</code>
<code>revenue_won</code>	sum of value of leads won
<code>emails_sent</code>	email sent from the app
<code>tasks_completed</code>	task set to <code>Done</code>
<code>active_hours</code>	active time from tracking

Flow:

```
Manager: Targets -> New target
  assignee: employee | team | role      period: daily | weekly | monthly
  metric + target value + valid from/to + active weekdays (daily only)
-> each metric event updates target_progress for the current period
-> desktop dashboard shows progress bars in real time
-> at 80% and 100%: notification to employee; at period end below target:
    listed in manager report
-> period end: snapshot row written (value, target, percent), new period starts
```

Rules:

- Period boundaries use the company timezone and configured week start. For night-shift employees the daily period follows the attendance date (F8), not midnight.
- A team target is the sum of member values. A role target applies individually to each employee with that role.
- An employee-level target overrides a role-level target for the same metric and period.
- Events are attributed to the acting employee at event time. Reassigning a lead later does not move past counts.
- Daily targets are not expected on leave days, holidays and weekly off days (shown as not applicable).

F14 - Email

Full design in 10. Flow summary:

```

Connect:  Admin -> Email -> Accounts -> Connect mailbox
          -> Google sign-in as that mailbox (internal OAuth app)
          -> backend stores encrypted refresh token, starts watch + first sync
Assign:   Admin -> Assign Accounts -> employee + mailbox + permissions
          (read, send, reply, draft, attach, download attachments,
           archive, mark read) -> employee notified
Read:     App -> Email -> mailbox -> thread list (from server cache)
          -> open thread -> server fetches body from Gmail -> sanitized HTML
Send:     App -> compose / reply -> POST to server -> permission check
          -> server sends through Gmail API -> audit entry
          -> if recipient matches a lead: timeline entry + emails_sent metric
New mail: Gmail -> Pub/Sub -> server sync -> WebSocket "mail.new" -> badge
Remove:   Unassign or disable employee -> access gone immediately,
          nothing to revoke at Google because the employee never had a token

```

Rules: employees never receive a Google credential or token. Delete, settings, filters, forwarding and password actions do not exist in the API. Every read, send and download is written to the email audit log with the employee id.

F15 - Notifications

Event	Recipient	Channel
Lead assigned or reassigned	New owner, previous owner	In-app, toast
Task assigned, due soon, overdue	Assignee, creator (overdue)	In-app, toast
Follow-up due	Lead owner	In-app, toast
Target 80 percent, reached	Employee	In-app
New email in assigned mailbox	Assigned employees	Badge, optional toast
Device pending approval	Approvers	In-app (admin), email
Correction or leave requested, decided	Manager, employee	In-app, toast
Shift auto-closed	Employee, manager	In-app
Security alert (lockout, token reuse, tamper)	Admins with <code>security.view</code>	In-app (admin), email
Announcement from admin	Selected employees	In-app, toast

Delivery: server stores the notification, pushes over WebSocket if online, otherwise delivered at next connect. Read state per recipient. Toasts respect a per-user mute setting except security and shift messages.

F16 - Offboarding

```

Admin -> Employee -> Disable
-> dialog lists owned open items: leads, tasks, follow-ups, appointments
-> admin selects new owner(s) (required when open items exist)
-> confirm
-> one transaction:
    status = disabled
    all sessions and refresh tokens revoked
    devices set to revoked
    mailbox assignments removed
    open shift auto-closed
    open items reassigned, timeline entries written
    targets ended
-> WebSocket session.revoked -> app logs out, agent stops
-> audit log entry

```

Rules: employee records are never hard-deleted (history and audit keep referencing them).
Re-enable is possible and requires new device approval. The employee code is not reused.
Reminder shown to the admin: also suspend the employee's own Google Workspace account, if they have one, in the Google Admin console.

F17 - Lost, stolen or replaced device

```
Admin -> Security -> Devices -> device -> Revoke
-> device status = revoked, its sessions revoked, push session.revoked
-> next login from that PC fails ("device not authorized")
Replacement PC -> F2 (new device pending -> approval)
```

Local data on the PC is limited to the encrypted segment queue and cached lists. On revoke with the app online, the app wipes its local cache and tokens.

F18 - Desktop auto-update

```
App checks updates.<domain> at start and every 4 hours
-> newer version for this release channel (pilot | stable)?
-> download in background, verify code signature
-> no open shift: install on next app start
    open shift:    install at End Shift, or at next start
-> version flagged "mandatory": app blocks Start Shift until updated
Agent and watchdog are shipped inside the same installer and updated together.
```

Server keeps `min_supported_version`. Older clients receive HTTP 426 and are forced to update.

F19 - Viewing employee data (admin side)

```
Manager -> Workforce -> Activity -> employee + date
-> permission + scope check (own team only for Team Leader)
-> day summary from rollups -> drill down: app -> domain -> time ranges
-> every view of another person's activity detail is written to the audit log
```

F20 - Daily journey summary

```
Employee: open app -> auto login -> Start Shift -> dashboard
-> works leads, calls, emails, tasks (targets update live)
-> break / resume -> End Shift -> day summary shown -> close app
Manager: Admin Panel dashboard -> who is working / late / absent
-> team targets -> overdue follow-ups -> approvals -> reports
```

04 - Module Specification

Functional requirements per module. Priority: M = MUST (1.0), S = SHOULD (1.0 if time allows), L = later.

1. Authentication (AUTH)

ID	Requirement	P
FR-AUTH-01	Employee login with employee code or company email + password	M
FR-AUTH-02	Passwords hashed with Argon2id	M
FR-AUTH-03	Access token 15 min (JWT), rotating refresh token 30 days bound to device	M
FR-AUTH-04	Refresh requests signed with the device private key	M
FR-AUTH-05	Lockout after 5 failed attempts in 15 min, rate limit per IP	M
FR-AUTH-06	Forced password change on temporary password	M
FR-AUTH-07	Admin Panel login with mandatory TOTP 2FA and recovery codes	M
FR-AUTH-08	Session list and revoke (per session, per employee, all)	M
FR-AUTH-09	Refresh token reuse detection revokes the session family	M
FR-AUTH-10	Login history with time, IP, device, result	M
FR-AUTH-11	Optional IP allowlist for the Admin Panel	S
FR-AUTH-12	Google sign-in for employees	L

2. Devices (DEV)

ID	Requirement	P
FR-DEV-01	Device registers on first login with public key and device info	M
FR-DEV-02	Approval modes manual, auto first device, off	M
FR-DEV-03	Approve, reject, revoke, rename devices in the Admin Panel	M
FR-DEV-04	Limit of approved devices per employee	M
FR-DEV-05	Device list shows last seen, app version, agent version, OS	M
FR-DEV-06	Local cache and tokens wiped on revoke	M

3. Organisation and employees (ORG)

ID	Requirement	P
FR-ORG-01	CRUD departments and teams, team has a leader, department has a manager	M
FR-ORG-02	CRUD employees: code, name, email, phone, department, team, role, manager, schedule, join date, status	M
FR-ORG-03	Employee statuses invited, active, disabled	M
FR-ORG-04	Reset password, disable (F16), re-enable	M
FR-ORG-05	Employee profile page with tabs: overview, attendance, activity, CRM, targets, devices, mail, audit	M
FR-ORG-06	Bulk import of employees from CSV	S

4. Roles and permissions (RBAC)

ID	Requirement	P
FR-RBAC-01	Permission keys with scope own, team, department, all (see 05)	M
FR-RBAC-02	Five default roles, editable, plus custom roles	M
FR-RBAC-03	Server enforces permission and scope on every endpoint. UI hiding is cosmetic only	M
FR-RBAC-04	Super Admin role cannot be edited or removed, at least one must exist	M
FR-RBAC-05	A user cannot grant permissions they do not hold	M

5. Shift and attendance (ATT)

ID	Requirement	P
FR-ATT-01	Start, break (with type), resume, end shift from the Desktop App	M
FR-ATT-02	Heartbeat, offline gap detection, auto-close per F5	M
FR-ATT-03	Shift schedules: name, workdays, start, end, grace minutes, may cross midnight	M
FR-ATT-04	Assign schedule per employee with effective dates	M
FR-ATT-05	Holidays calendar	M
FR-ATT-06	Attendance day status per F8, recomputed on corrections	M
FR-ATT-07	Live attendance board: status, check-in, active time, current app category	M
FR-ATT-08	Correction requests with approval (F9)	M
FR-ATT-09	Leave requests with approval (F9)	S
FR-ATT-10	Monthly attendance sheet per employee and per team, CSV export	M
FR-ATT-11	Long idle prompt (F5 rule 6)	S

6. Activity tracking (TRK)

ID	Requirement	P
FR-TRK-01	Record active, idle and locked time per shift	M
FR-TRK-02	Record time per application (process and display name)	M
FR-TRK-03	Record time per website domain for Chrome and Edge through the extension	M
FR-TRK-04	Record keyboard and mouse event counts per segment, never content	M
FR-TRK-05	Activity exceptions for call and meeting apps and microphone use	M
FR-TRK-06	Local encrypted queue, idempotent batch upload	M
FR-TRK-15	Offline-first: tracking, shift timer, break and end shift keep working with no internet. Queue syncs automatically on reconnect and the Admin Panel backfills the full offline period (F7)	M
FR-TRK-07	Daily rollups per employee, app, domain, category	M
FR-TRK-08	Drill-down: day, app, domain, time ranges (as in the original example)	M
FR-TRK-09	Productivity categories for apps and domains, default + per department override	M
FR-TRK-10	Uncategorised apps and domains queue for admin classification	S
FR-TRK-11	Tracking exclusions list	M
FR-TRK-12	Tamper signals: agent stopped, extension missing, clock changed	M
FR-TRK-13	Employee sees own activity summary in the app	S
FR-TRK-14	Screenshots, optional, configurable interval, blur option	L

7. CRM (CRM)

ID	Requirement	P
FR-CRM-01	Lead fields: name, company, phones, emails, source, owner, pipeline, stage, value, tags, follow-up date, custom fields	M
FR-CRM-02	Pipelines and stages configurable, stage type open, won, lost, metric flags	M
FR-CRM-03	List view with filters, search, saved views. Board (kanban) view per pipeline	M (list), S (board)
FR-CRM-04	Timeline per lead, append-only	M
FR-CRM-05	Notes, attachments (max 25 MB, type allowlist)	M
FR-CRM-06	Call logging per F12	M
FR-CRM-07	Appointments: date-time, title, notes, reminder	M
FR-CRM-08	Follow-up reminders and "Follow-ups Today" list	M
FR-CRM-09	Duplicate detection on phone and email	M
FR-CRM-10	CSV import with mapping, preview, round-robin assignment	M
FR-CRM-11	Reassign single and bulk	M
FR-CRM-12	Lost reason required, list configurable	M
FR-CRM-13	Won lead flagged as client, Clients view	M
FR-CRM-14	Export CSV, permission-gated and audited	M
FR-CRM-15	Custom fields (text, number, date, select) defined by admin	S
FR-CRM-16	Emails exchanged with a lead address appear in its timeline	S

8. Tasks (TASK)

ID	Requirement	P
FR-TASK-01	Create, assign, prioritise, due date, optional lead link	M
FR-TASK-02	Status flow To do, In progress, Done, Cancelled	M
FR-TASK-03	Reminders and overdue notifications	M
FR-TASK-04	My Tasks view grouped by today, upcoming, overdue, done	M
FR-TASK-05	Comments on tasks	S

9. Targets (TGT)

ID	Requirement	P
FR-TGT-01	Metric catalog per F13	M
FR-TGT-02	Targets per employee, team, role and period	M
FR-TGT-03	Real-time progress in the app and Admin Panel	M
FR-TGT-04	Period snapshots and history	M
FR-TGT-05	Threshold notifications	S
FR-TGT-06	Leaderboard per team and metric	S

10. Email (MAIL)

ID	Requirement	P
----	-------------	---

FR-MAIL-01	Connect, reconnect and disconnect Workspace mailboxes	M
FR-MAIL-02	Assign mailbox to employees with per-assignment permissions	M
FR-MAIL-03	Thread list, thread view, search, unread state	M
FR-MAIL-04	Compose, reply, reply all, forward, drafts, attachments	M
FR-MAIL-05	HTML sanitised, remote images blocked until the user allows	M
FR-MAIL-06	Near real-time new mail through Pub/Sub, fallback polling	M
FR-MAIL-07	Email audit log	M
FR-MAIL-08	No delete, no settings, no forwarding rules, no filters exposed	M
FR-MAIL-09	Signature per mailbox and per employee	S
FR-MAIL-10	Mailbox health status and alert when a token is revoked	M

11. Notifications (NOTIF)

ID	Requirement	P
FR-NOTIF-01	Stored notifications with read state, list and badge	M
FR-NOTIF-02	WebSocket push, Windows toast in the app	M
FR-NOTIF-03	Admin announcements to selected employees	S
FR-NOTIF-04	Email channel for security alerts to admins	M

12. Reports (RPT)

ID	Report	Content	P
FR-RPT-01	Attendance	Per day, month, employee, team. Late, absent, half day counts	M
FR-RPT-02	Time	Working, active, idle, break, offline per employee and period	M
FR-RPT-03	Applications and websites	Top apps and domains, per employee, team, company	M
FR-RPT-04	Productivity	Productive, neutral, unproductive share and trend	M
FR-RPT-05	CRM	Leads by stage, source, owner. Conversion, overdue follow-ups	M
FR-RPT-06	Calls	Calls and connect rate per employee and period	M
FR-RPT-07	Targets	Achievement per employee, team, period	M
FR-RPT-08	CSV export for every report, audited	M	
FR-RPT-09	Scheduled reports by email	L	

13. Audit and security (AUD)

ID	Requirement	P
FR-AUD-01	Append-only audit log: actor, action, entity, before and after, IP, device, time	M
FR-AUD-02	Logged: all admin writes, permission changes, logins, device actions, exports, viewing another person's activity detail, mail access	M
FR-AUD-03	Audit log viewer with filters, export	M
FR-AUD-04	Security alerts list	M

14. Settings (SET)

Company profile, timezone, week start, idle threshold, long idle prompt, break types and allowance, auto-close limits, device approval mode, device limit, tracking options (titles, exclusions, exception apps), productivity categories, lost reasons, lead sources, notification defaults, retention periods, monitoring policy text and version, release channel per device.

15. Desktop App screens

Screen	Content
Login	Code or email, password, device pending state
Consent	Policy text, accept or decline
Dashboard	Greeting, shift control and timer, working time, targets, tasks summary, follow-ups today, new mail count
My Tasks	Grouped list, detail drawer
CRM	Leads list, lead detail with timeline, log call, note, email, task, appointment
Email	Mailbox switcher, thread list, thread view, compose
Targets	Daily, weekly, monthly progress, history
Attendance	Month calendar, day detail, correction and leave requests
Activity	Own day summary (if enabled)
Notifications	List
Profile	Details, change password, device info, policy
Tray	Status, Start or End Shift, Break, Open, Quit

A visible indicator (tray icon colour and header badge) shows when tracking is running.

16. Admin Panel screens

Structure follows the original proposal with the additions marked (+).

Dashboard	live board, KPIs, alerts, approvals (+)
Employees	Employees, Departments, Teams (+), Roles, Permissions, Devices
CRM	Leads, Clients, Tasks, Pipelines, Import (+), Reports
Workforce	Attendance, Schedules (+), Holidays (+), Leave and Corrections (+), Time Tracking, Activity, Applications, Websites, Productivity Rules (+)
Targets	Targets, Metrics, History (+), Leaderboard (+)
Email	Accounts, Assign Accounts, Permissions, Email Logs
Reports	Attendance, Time, Productivity, CRM, Calls (+), Targets
Security	Login History, Devices, Sessions, IP Logs, Audit Logs, Alerts (+), Access Control
Settings	Company, Tracking, Attendance rules, Notifications, Retention (+), Policy text (+), Releases (+)

05 - Roles and Permissions

1. Model

- A permission is a key such as `crm.leads.view`.
- A role is a named set of permissions, each granted with a scope.
- Each employee has exactly one role. Individual overrides (grant or deny a key) are possible and audited.
- Scope defines whose data the permission reaches.

Scope	Reaches
own	Records owned by or assigned to the user
team	Records of members of teams the user leads, plus own
department	Records of all employees in departments the user manages, plus own
all	Everything in the company

Server check for every request: `has(permission)` AND `target within scope`. A request outside scope returns 404 for single records (do not reveal existence) and filters lists silently.

Access channels: the Desktop App is for every employee. The Admin Panel needs the permission `admin.access` and 2FA.

2. Permission catalog

Group	Keys
Admin	<code>admin.access</code>
Employees	<code>employees.view</code> , <code>employees.create</code> , <code>employees.edit</code> , <code>employees.disable</code> , <code>employees.reset_password</code>
Organisation	<code>org.manage</code> (departments, teams)
Roles	<code>roles.view</code> , <code>roles.manage</code>
Devices	<code>devices.view</code> , <code>devices.approve</code> , <code>devices.revoke</code>
Sessions	<code>sessions.view</code> , <code>sessions.revoke</code>
Attendance	<code>attendance.view</code> , <code>attendance.manage</code> (schedules, holidays), <code>attendance.approve</code> (corrections, leave)
Tracking	<code>tracking.view_summary</code> (totals), <code>tracking.view_detail</code> (apps, domains, time ranges), <code>tracking.manage_rules</code>
CRM leads	<code>crm.leads.view</code> , <code>crm.leads.create</code> , <code>crm.leads.edit</code> , <code>crm.leads.assign</code> , <code>crm.leads.reopen</code> , <code>crm.leads.delete</code> , <code>crm.leads.import</code> , <code>crm.leads.export</code> , <code>crm.leads.create_duplicate</code>
CRM config	<code>crm.pipelines.manage</code>
Calls	<code>crm.calls.log</code> , <code>crm.calls.manage</code>
Tasks	<code>tasks.view</code> , <code>tasks.create</code> , <code>tasks.assign</code> , <code>tasks.manage</code>
Targets	<code>targets.view</code> , <code>targets.manage</code>
Mail	<code>mail.use</code> (assigned mailboxes), <code>mail.accounts.manage</code> , <code>mail.assign</code> , <code>mail.logs.view</code>
Reports	<code>reports.view</code> , <code>reports.export</code>
Notifications	<code>announcements.send</code>

Security	<code>security.view</code> (login history, alerts), <code>audit.view</code> , <code>audit.export</code>
Settings	<code>settings.manage</code>

Keys live in `packages/contracts` as a typed constant. Adding a key requires a migration that updates the default roles.

3. Default role matrix

Cell value = scope. Empty = not granted.

Permission	Employee	Team Leader	Manager	Admin	Super Admin
<code>admin.access</code>		yes	yes	yes	yes
<code>employees.view</code>		team	department	all	all
<code>employees.create / edit</code>				all	all
<code>employees.disable / reset_password</code>				all	all
<code>org.manage</code>				all	all
<code>roles.view</code>				all	all
<code>roles.manage</code>					all
<code>devices.view</code>			department	all	all
<code>devices.approve / revoke</code>				all	all
<code>sessions.view / revoke</code>				all	all
<code>attendance.view</code>	own	team	department	all	all
<code>attendance.approve</code>		team	department	all	all
<code>attendance.manage</code>				all	all
<code>tracking.view_summary</code>	own	team	department	all	all
<code>tracking.view_detail</code>	own		department	all	all
<code>tracking.manage_rules</code>				all	all
<code>crm.leads.view / edit</code>	own	team	department	all	all
<code>crm.leads.create</code>	own	team	department	all	all
<code>crm.leads.assign</code>		team	department	all	all
<code>crm.leads.reopen</code>		team	department	all	all
<code>crm.leads.delete</code>				all	all
<code>crm.leads.import / export</code>			department	all	all
<code>crm.pipelines.manage</code>				all	all
<code>crm.calls.log</code>	own	team	department	all	all
<code>crm.calls.manage</code>		team	department	all	all
<code>tasks.view</code>	own	team	department	all	all
<code>tasks.create</code>	own	team	department	all	all
<code>tasks.assign / manage</code>		team	department	all	all
<code>targets.view</code>	own	team	department	all	all
<code>targets.manage</code>			department	all	all
<code>mail.use</code>	own	own	own	own	own
<code>mail.accounts.manage</code>					all
<code>mail.assign</code>				all	all

mail.logs.view				all	all
reports.view		team	department	all	all
reports.export			department	all	all
announcements.send		team	department	all	all
security.view				all	all
audit.view				all	all
audit.export					all
settings.manage				all	all

Notes:

1. Team Leaders see tracking totals of their team but not the app and website detail by default. This is a privacy-minded default and can be changed per role.
2. `mail.use` always has scope `own`: a user sees only mailboxes assigned to them, regardless of role. Admins manage assignments but do not read mail unless a mailbox is assigned to them.
3. `mail.accounts.manage` (connecting a mailbox) is Super Admin only by default because it creates access to a Google mailbox.
4. Lead deletion is a soft delete and restricted to Admin.

4. Rules

ID	Rule
R1	Super Admin role is fixed and has every permission with scope all
R2	At least one active Super Admin must exist. The last one cannot be disabled or demoted
R3	A user cannot edit their own role or permissions
R4	A user can only grant permissions and scopes they hold themselves
R5	A user cannot act on an employee with a higher role rank (rank: Employee 1, Team Leader 2, Manager 3, Admin 4, Super Admin 5, custom roles carry a rank)
R6	Permission and role changes take effect on the next request (permissions are loaded per request from cache, invalidated on change)
R7	Every permission or role change is written to the audit log with before and after
R8	Tracking data of Admin and Super Admin users is visible only to Super Admin

5. Implementation notes

- NestJS: `@RequirePermission('crm.leads.view')` decorator + global guard. The guard resolves the scope and attaches a `ScopeFilter` to the request. Every repository query MUST apply that filter. No query on employee-owned data without it.
- Scope resolution: `own -> employee_id = me`. `team -> employee_id IN members(teams led by me) + me`. `department -> employee_id IN employees(departments managed by me) + me`. `all -> company_id` only.
- Membership sets are cached in Redis per user for 60 s and invalidated on org changes.
- Tests: for every endpoint one test per role proving allowed and denied access (see 11).

06 - Data Model

PostgreSQL 16. Conventions for every table unless stated:

- `id uuid` primary key (UUID v7, generated by the server).
- `company_id uuid not null` with index. Every query filters by it.
- `created_at, updated_at timestamptz not null`. Soft delete through `deleted_at timestamptz null` where marked (SD).
- Enums are stored as text with check constraints, defined once in `packages/contracts`.
- Money as `numeric(14,2)` + currency code. Durations in seconds as `integer`.
- Foreign keys always indexed.

1. Identity and organisation

Table	Key columns
companies	name, timezone, week_start, settings jsonb
departments	name, manager_id -> employees
teams	name, department_id, leader_id -> employees
employees (SD)	code (unique per company), first_name, last_name, email (unique per company, null allowed), phone, password_hash, must_change_password, status (invited, active, disabled), role_id, department_id, team_id, manager_id, join_date, totp_secret_enc, totp_enabled, failed_logins, locked_until, last_login_at, disabled_at
roles	name, rank, is_system
role_permissions	role_id, permission_key, scope. Unique (role_id, permission_key)
employee_permission_overrides	employee_id, permission_key, effect (grant, deny), scope
devices	employee_id, name, public_key, hardware_hash, os_version, app_version, agent_version, status (pending, approved, rejected, revoked), approved_by, approved_at, last_seen_at, release_channel
sessions	employee_id, device_id (null for admin web), family_id, refresh_token_hash, client (desktop, admin), ip, user_agent, expires_at, revoked_at, revoked_reason
login_events	employee_id null, identifier, device_id null, ip, client, result (success, bad_password, locked, device_pending, device_revoked, disabled, 2fa_failed), created_at
consent_records	employee_id, policy_version, accepted_at, device_id, ip
policy_versions	version, text, published_at

2. Attendance

Table	Key columns
shift_schedules	name, workdays int[] (1 to 7), start_time, end_time, crosses_midnight, grace_minutes, half_day_percent
employee_schedules	employee_id, schedule_id, effective_from, effective_to
holidays	date, name
leave_requests	employee_id, type, date_from, date_to, reason, status (pending, approved, rejected, cancelled), decided_by, decided_at, comment

shifts	employee_id, device_id, attendance_date, started_at, ended_at, state (working, on_break, ended), end_reason (manual, auto_closed, recovered, admin), last_heartbeat_at, flags text[], totals: duration_s, break_s, offline_s, active_s, idle_s
shift_events	shift_id, client_event_id uuid unique, type (start, break_start, break_end, end), break_type, occurred_at, received_at, reconstructed bool
shift_breaks	shift_id, type, started_at, ended_at
attendance_days	employee_id, date, schedule_id, status, check_in_at, check_out_at, working_s, active_s, idle_s, break_s, offline_s, flags text[], computed_at. Unique (employee_id, date)
attendance_corrections	employee_id, date, type, requested_start, requested_end, reason, status, decided_by, decided_at, comment
attendance_adjustments	attendance_day_id, correction_id, delta fields, created_by

Constraint: partial unique index on shifts (employee_id) WHERE state <> 'ended' guarantees one open shift.

3. Tracking

Table	Key columns
activity_segments	id (client UUID), employee_id, device_id, shift_id, started_at, ended_at, duration_s, kind (active, idle, locked), process_name, app_name, domain null, window_title null, key_count, mouse_count, exception (none, call_app, meeting_app, mic), claim (none, break, away_work), received_at, late bool
activity_daily_app	employee_id, date, app_name, process_name, category, active_s, idle_s. Unique (employee_id, date, process_name)
activity_daily_domain	employee_id, date, domain, category, active_s. Unique (employee_id, date, domain)
activity_daily_summary	employee_id, date, tracked_s, active_s, idle_s, productive_s, neutral_s, unproductive_s, key_count, mouse_count
productivity_rules	target_type (app, domain), pattern, category (productive, neutral, unproductive), department_id null (null = company default)
tracking_exclusions	target_type, pattern
activity_exception_apps	process_name, type (call, meeting)
tamper_events	employee_id, device_id, type (agent_stopped, extension_missing, clock_changed, binary_mismatch), detail jsonb, occurred_at
screenshots (1.1)	employee_id, shift_id, taken_at, object_key, blurred

activity_segments is range-partitioned by month on started_at. Indexes: (employee_id, started_at), (shift_id). Primary key (id, started_at). Insert is ON CONFLICT DO NOTHING on id for idempotency. date in rollups is the attendance date.

4. CRM and tasks

Table	Key columns
pipelines	name, is_default
pipeline_stages	pipeline_id, name, position, type (open, won, lost), counts_as_qualified, counts_as_meeting
leads (SD)	name, company_name, owner_id, pipeline_id, stage_id, source_id, value, currency, follow_up_at, do_not_call, is_client, won_at, lost_at, lost_reason_id, custom jsonb, qualified_at, last_activity_at,

	created_by
lead_phones	lead_id, phone_e164, label, is_primary. Index on phone_e164
lead_emails	lead_id, email_lower, label, is_primary. Index on email_lower
lead_sources, lost_reasons, tags, lead_tags	lookup tables
lead_timeline	lead_id, actor_id, type (created, stage_changed, assigned, note, call, email, task, appointment, attachment, imported), ref_id, data jsonb, occurred_at. Append-only
lead_notes	lead_id, author_id, body
call_logs	lead_id, employee_id, direction, outcome, duration_s, notes, source (manual, integration), occurred_at
appointments	lead_id, employee_id, title, starts_at, ends_at, notes, status (scheduled, done, cancelled, no_show)
tasks (SD)	title, description, assignee_id, creator_id, lead_id null, priority, due_at, status, completed_at
task_comments	task_id, author_id, body
attachments	owner_type (lead, task), owner_id, uploader_id, file_name, mime, size, object_key
custom_field_defs	entity (lead), key, label, type, options jsonb, position
import_jobs	uploader_id, file_key, mapping jsonb, status, totals jsonb, report_key

5. Targets

Table	Key columns
targets	assignee_type (employee, team, role), assignee_id, metric_key, period (daily, weekly, monthly), value, weekdays int[] null, valid_from, valid_to, created_by
metric_events	employee_id, metric_key, amount, occurred_at, period_date (attendance date), ref_type, ref_id. Unique (metric_key, ref_type, ref_id) to prevent double counting
target_progress	target_id, employee_id, period_start, period_end, value, target_value, closed bool. Unique (target_id, employee_id, period_start)

6. Email

Table	Key columns
mailboxes	address, display_name, google_user_id, refresh_token_enc, token_status (ok, revoked, error), history_id, watch_expires_at, last_sync_at, signature_html, connected_by
mailbox_assignments	mailbox_id, employee_id, permissions text[] (read, send, reply, draft, attach, download, archive, mark_read), assigned_by. Unique (mailbox_id, employee_id)
mail_threads	mailbox_id, gmail_thread_id, subject, snippet, participants jsonb, last_message_at, unread, has_attachments, label_ids text[], lead_id null. Unique (mailbox_id, gmail_thread_id)
mail_messages	thread_id, gmail_message_id, from_addr, to_addrs, cc_addrs, sent_at, snippet, has_attachments, sent_by_employee_id null. Body is not stored (fetched on demand)
mail_audit	mailbox_id, employee_id, action (list, open, send, reply, forward, draft, download_attachment, archive, mark_read), gmail_message_id, recipients, subject, occurred_at, ip, device_id

7. Platform

Table	Key columns
notifications	recipient_id, type, title, body, data jsonb, read_at
audit_logs	actor_id, actor_role, action, entity_type, entity_id, before jsonb, after jsonb, ip, device_id, client, occurred_at. Append-only, partitioned by month
security_alerts	type, severity, employee_id null, detail jsonb, acknowledged_by, acknowledged_at
app_releases	version, channel (pilot, stable), mandatory, min_supported, notes, installer_key, signature, published_at
settings	key, value jsonb (company-level overrides of defaults)

`audit_logs` and `lead_timeline`: the application database role has INSERT and SELECT only. No UPDATE or DELETE grants.

8. Retention defaults

Data	Retention	Configurable
activity_segments (raw)	90 days	yes
Daily rollups and attendance	Unlimited	yes
window_title, page title (if enabled)	30 days	yes
Screenshots (1.1)	30 days	yes
login_events	1 year	yes
audit_logs, mail_audit	2 years	yes
notifications	90 days	yes
CRM data	Unlimited	no

9. Key relations

```

company 1-n department 1-n team 1-n employee
employee 1-n device 1-n session
employee 1-n shift 1-n shift_break
shift 1-n activity_segment -> daily rollups (employee, date)
employee 1-n attendance_day
employee 1-n lead (owner) 1-n timeline / call_log / note / appointment / task
target n-1 assignee (employee | team | role) ; target 1-n target_progress
mailbox n-n employee (mailbox_assignments) ; mailbox 1-n thread 1-n message

```

07 - API Specification

This file defines conventions and the endpoint inventory. Exact request and response shapes are the Zod schemas in `packages/contracts`. OpenAPI is generated from them and is the binding reference once code exists.

1. Conventions

Topic	Rule
Base URL	<code>https://api.<domain>/api/v1</code>
Format	JSON, UTF-8, camelCase field names, ISO 8601 UTC timestamps
Auth, desktop	Authorization: Bearer <access token> + X-Device-Id
Auth, admin	Session cookie + CSRF token header on writes
Pagination	Cursor based: <code>?limit=50&cursor=...</code> -> <code>{ items, nextCursor }</code> . Max limit 200
Filtering	Query parameters, documented per endpoint. Dates as <code>from</code> , <code>to</code>
Idempotency	Writes from the desktop carry Idempotency-Key (UUID). Server stores result for 24 h
Versioning	Path version. X-Client-Version header on every request. Below minimum -> 426
Rate limits	Login 10 per minute per IP. General 300 per minute per session. Ingest 30 per minute per device
Time	Every response includes header X-Server-Time

Error body:

```
{
  "error": {
    "code": "SHIFT_ALREADY_OPEN",
    "message": "A shift is already open on another device.",
    "details": { "shiftId": "...", "deviceName": "PC-014" },
    "requestId": "..."
  }
}
```

HTTP	Use
400	Validation failed (VALIDATION_ERROR with field list)
401	Not authenticated, token expired, session revoked
403	Authenticated but permission missing
404	Not found or outside scope
409	State conflict
422	Business rule violated
426	Client version too old
429	Rate limited

2. Auth and devices

Method	Path	Purpose
POST	/auth/login	Desktop login. Body: identifier, password, device {id?, publicKey, name, os, hardwareHash}, signature. Returns tokens, or

		DEVICE_PENDING
GET	/auth/device-status	Poll while device is pending
POST	/auth/refresh	Rotating refresh, signed by device key
POST	/auth/logout	Revoke current session
POST	/auth/change-password	Current + new password
POST	/auth/consent	Accept policy version
GET	/me	Profile, permissions with scopes, settings relevant to the client, open shift
POST	/admin/auth/login, /admin/auth/2fa, /admin/auth/logout	Admin Panel login
POST	/admin/auth/2fa/enroll, /admin/auth/2fa/verify	TOTP enrolment
GET	/devices	List (scope)
POST	/devices/:id/approve, /reject, /revoke	Device actions
GET	/sessions, DELETE /sessions/:id	Session management
GET	/login-events	Login history

3. Organisation

Method	Path
GET, POST	/employees
GET, PATCH	/employees/:id
POST	/employees/:id/reset-password
GET	/employees/:id/offboarding-preview
POST	/employees/:id/disable (body: reassignment map), /employees/:id/enable
CRUD	/departments, /teams
CRUD	/roles, PUT /roles/:id/permissions
GET	/permissions (catalog)

4. Shift and attendance

Method	Path	Purpose
POST	/shifts/events	Batch of shift events (start, break_start, break_end, end), each with clientEventId, monotonic offset. Used online and for offline replay
POST	/shifts/heartbeat	State, client clock, queue size. Response: server time, config version, commands
GET	/shifts/current	Open shift
GET	/attendance/days	Filter employee, team, from, to
GET	/attendance/live	Live board
CRUD	/schedules, /employee-schedules, /holidays	

POST, GET	/attendance/corrections, POST /attendance/corrections/:id/decide	
POST, GET	/leave-requests, POST /leave-requests/:id/decide	

5. Tracking

Method	Path	Purpose
POST	/tracking/segments	Batch ingest, max 500
POST	/tracking/tamper-events	Tamper signals
GET	/tracking/config	Idle threshold, exception apps, exclusions, capture flags
GET	/tracking/summary	Totals per employee and date range
GET	/tracking/apps, /tracking/domains	Rollups with drill-down parameters
GET	/tracking/timeline	Segments for one employee and day (permission tracking.view_detail, audited)
CRUD	/tracking/productivity-rules, /tracking/exclusions, /tracking/exception-apps	
GET	/tracking/uncategorised	Apps and domains without a rule

Ingest example:

```

POST /tracking/segments
{
  "shiftId": "0192...",
  "segments": [
    {
      "id": "0192f3a1-...",
      "startedAt": "2026-10-03T05:20:00Z",
      "endedAt": "2026-10-03T05:24:10Z",
      "kind": "active",
      "processName": "chrome.exe",
      "appName": "Google Chrome",
      "domain": "youtube.com",
      "keyCount": 12,
      "mouseCount": 240,
      "exception": "none",
      "clockSource": "anchored"
    }
  ]
}
-> 200 { "accepted": ["0192f3a1-..."], "rejected": [] }

```

Rejected items carry a reason (`OUTSIDE_SHIFT`, `OVERLAP`, `T00_OLD`, `INVALID`). The client deletes accepted and permanently rejected segments from its queue and keeps the rest for retry.

6. CRM, tasks, calls

Method	Path
GET, POST	/leads (filters: owner, stage, pipeline, source, tag, followUp, search)
GET, PATCH, DELETE	/leads/:id
POST	/leads/:id/stage, /leads/:id/assign, /leads/bulk-assign
GET	/leads/:id/timeline

POST	/leads/:id/notes, /leads/:id/calls, /leads/:id/appointments, /leads/:id/attachments
POST	/leads/check-duplicate
POST	/leads/import (upload), GET /leads/import/:jobId
POST	/leads/export (creates a job, returns download link)
CRUD	/pipelines, /pipelines/:id/stages, /lead-sources, /lost-reasons, /tags, /custom-fields
GET, POST	/tasks, GET, PATCH /tasks/:id, POST /tasks/:id/comments
GET	/follow-ups (today, overdue, upcoming)
POST	/files/presign (upload URL), GET /files/:id (download redirect, permission checked)

7. Targets

Method	Path
GET	/metrics
CRUD	/targets
GET	/targets/progress (mine or by employee, team, period)
GET	/targets/history, /targets/leaderboard

8. Mail

Method	Path	Purpose
GET	/mail/oauth/start, /mail/oauth/callback	Connect mailbox
GET, DELETE	/mailboxes, /mailboxes/:id	Manage
CRUD	/mailboxes/:id/assignments	Assign with permissions
GET	/mail/my-mailboxes	Mailboxes assigned to the caller with permissions
GET	/mail/:mailboxId/threads	List, search q, label filter
GET	/mail/:mailboxId/threads/:threadId	Messages with sanitised bodies
POST	/mail/:mailboxId/send	New, reply, forward. Attachments by file ids
POST, PUT, DELETE	/mail/:mailboxId/drafts	Drafts
POST	/mail/:mailboxId/threads/:threadId/mark-read, /archive	State
GET	/mail/:mailboxId/messages/:id/attachments/:attId	Download (permission download)
POST	/mail/google/push	Pub/Sub webhook, verified by Google OIDC token
GET	/mail/audit	Email logs

9. Notifications, reports, audit, settings, releases

Method	Path
GET	/notifications, POST /notifications/:id/read, POST /notifications/read-all
POST	/announcements
GET	/reports/attendance, /reports/time, /reports/apps, /reports/productivity, /reports/crm, /reports/calls, /reports/targets (each accepts format=csv)

GET	/dashboard/admin, /dashboard/me
GET	/audit-logs, /security-alerts, POST /security-alerts/:id/ack
GET, PATCH	/settings
GET	/releases/latest?channel= (desktop), CRUD /releases (admin)
GET	/health, /health/ready

10. WebSocket

Connection: `wss://api.<domain>/ws`, authenticated with the access token, re-authenticated on refresh. Rooms: user, team, company-admins.

Event (server -> client)	Payload	Client action
session.revoked	reason	Stop agent, wipe tokens, show login
config.updated	version	Refetch config
notification.new	notification	Badge, toast
mail.new	mailboxId, threadId	Refresh list, badge
target.progress	targetId, value	Update bars
lead.assigned	leadId	Refresh
task.updated	taskId	Refresh
device.approved	deviceId	Continue login
live.attendance	employeeId, status	Admin live board
update.available	version, mandatory	Start update flow

Clients treat WebSocket events as hints and refetch through REST. No business data is trusted from the socket alone.

08 - Security and Privacy

1. Threats considered

Threat	Main controls
Stolen or guessed employee password	Device approval, device-bound tokens, lockout, breached-password check
Token copied to another PC	Refresh requires signature by the device private key
Former employee keeps access	Offboarding transaction, immediate session revoke, no Google credentials ever issued
Employee exports the lead database	Scope limits, export permission, audit, rate limits on list endpoints
Employee stops or fakes tracking	Watchdog, tamper events, server-side validation, no local admin rights
Compromised admin account	Mandatory 2FA, IP allowlist option, audit log, rank rules
Backend compromise exposes mailboxes	Tokens encrypted with a key outside the database, per-mailbox OAuth (limited blast radius), minimal scopes
Malicious email content	HTML sanitising, sandboxed rendering, remote content blocked
Malicious update	Code signing, signature check before install
Insider views colleagues' data	Scopes, audit of detail views, restricted default for tracking detail

2. Authentication details

Item	Specification
Password hash	Argon2id, memory 64 MB, iterations 3, parallelism 1, per-user salt
Access token	JWT, EdDSA or RS256, 15 min, claims: sub, company, session, device, role version
Refresh token	256-bit random, stored hashed (SHA-256), rotating, 30 days sliding, bound to device and session family
Device key	ECDSA P-256 key pair generated on the PC. Private key non-exportable in the Windows key store (CNG, TPM-backed when available)
Proof	Login and refresh send <code>signature = sign(deviceKey, nonce + timestamp + body hash)</code> . Server verifies against the stored public key. Timestamp tolerance 2 min, nonce single use
Token storage, desktop	Electron <code>safeStorage</code> (DPAPI). Never in plain files, never in the renderer
Admin 2FA	TOTP (RFC 6238), 10 single-use recovery codes, secret encrypted at rest
Admin session	Opaque session id in <code>HttpOnly, Secure, SameSite=Strict</code> cookie. CSRF token on writes

3. Transport and data protection

- TLS 1.2 or higher everywhere. HSTS on all domains. The desktop app pins the API host name and rejects invalid certificates. No fallback to HTTP.
- Database on encrypted volumes. Backups encrypted.
- Field-level encryption (AES-256-GCM, key from the secret manager, key id stored with the value) for: Gmail refresh tokens, TOTP secrets.
- Object storage private. Access only through short-lived presigned URLs (5 min) issued after a permission check.

- Local agent queue: SQLite file encrypted with a random key protected by DPAPI for the user.
- Secrets never logged. Logs carry request id, user id, no passwords, tokens or mail content.

4. Application security checklist

Area	Requirement
Input	Zod validation on every endpoint, reject unknown fields
SQL	Prisma or parameterised SQL only
Authorisation	Global guard, scope filter mandatory in repositories, tests per role
Tenant isolation	company_id taken from the session, never from the request body
Uploads	Size limit, MIME allowlist, stored with random keys, served with Content-Disposition: attachment
Admin Panel	Strict CSP, no inline scripts, X-Frame-Options: DENY, dependency audit in CI
Electron	contextIsolation: true, sandbox: true, nodeIntegration: false, narrow preload API, navigation and new windows blocked, CSP, external links open in the system browser
Email HTML	Server-side sanitise (allowlist), render in a sandboxed iframe without scripts, remote images proxied or blocked until allowed, links show real target
Webhooks	Pub/Sub push verified through Google-signed OIDC token and audience
Rate limiting	Per IP, per session, per device, in Redis
Dependencies	Lockfiles, automated vulnerability scan, no unpinned install scripts
Errors	Generic messages to clients, details only in logs

5. Desktop and agent integrity

1. Installer and all binaries are signed with the company's code signing certificate (OV or EV). Unsigned monitoring software is commonly blocked by antivirus products and SmartScreen.
2. Installation is per machine and requires administrator rights. Employees run as standard users and cannot stop the watchdog service or replace files.
3. The watchdog verifies the signature of the agent before starting it and restarts it if it exits during a shift.
4. The named pipe is restricted by ACL to the logged-in user and both ends verify the peer process path and signature.
5. Tamper signals sent to the server: agent not running during an open shift, extension not connected while a browser is in the foreground, clock jump, binary signature mismatch, queue file modified.
6. The server validates plausibility: segments must lie within the shift, must not overlap, input counts must be within human limits. Anomalies raise a security alert. The server never trusts the client for attendance times.
7. Limits to state honestly: a user with local administrator rights can defeat any client-side tracker. The control for that is device management (standard user accounts), not software.

6. Monitoring transparency and legal basis

This system processes employee behavioural data. The legal requirements depend on where the employees are located and on the client's jurisdiction, and must be confirmed by the client's legal adviser (Q4). The design assumes transparent, proportionate monitoring:

Principle	Implementation
Notice	Written monitoring policy shown at first login and available in Profile. Version tracked, acceptance recorded
Work time only	Tracking runs only between Start Shift and End Shift and is paused during breaks
Visibility	Tray icon and header badge show when tracking is on. No hidden or stealth mode exists
Data minimisation	No keystroke content, no clipboard, no webcam, no audio. Domains instead of URLs. Window titles off by default. Counts instead of content
Access limitation	Detail data restricted by role and scope, every detail view audited
Own data	Employees can see their own tracked summary (Q10)
Retention	Raw data deleted after the retention period (06)
Exclusions	Domains such as banking or health portals can be excluded
Screenshots (1.1)	Off by default. If enabled: announced in policy, optional blur, short retention, access audited

Keyboard counting is implemented without reading key values (see 09). This is also what keeps the agent from behaving like, and being classified as, a keylogger.

7. Audit

Audit entries are append-only (database grants) and cover: authentication events, employee and role changes, permission changes, device and session actions, settings changes, target changes, lead reassignment, import, export, deletion, mailbox connection and assignment, every mail action, viewing another employee's activity detail, report export, attendance decisions and adjustments, release publishing.

Each entry: actor, role at the time, action, entity, before and after values (sensitive fields masked), IP, device, client, timestamp, request id.

8. Operations

Topic	Requirement
Backups	Daily full + point-in-time recovery, 30 days, stored in a second location, monthly restore test
Recovery targets	RPO 15 minutes, RTO 4 hours
Access to production	Named accounts, SSH keys or SSO, MFA, no shared passwords, access logged
Database access	API role with least privilege. Migrations run with a separate role
Monitoring	Error tracking, uptime, queue depth, failed login spikes, mailbox token failures
Incident handling	Security alerts to admins by email. Runbook: revoke sessions, rotate keys, disconnect mailboxes
Key rotation	JWT signing keys and field encryption keys rotatable (key id in data), rotate yearly or on incident
Pre-release	Dependency scan, OWASP ASVS level 2 self-check, external penetration test recommended before full rollout

09 - Tracker Agent, Watchdog and Browser Extension

1. Processes

Process	Runs as	Session	Lifetime	Job
CompanyOS.Agent.exe	Logged-in user	User session	From logon, idle until told to track	Measure activity, queue segments, host native messaging
CompanyOS.Watchdog.exe	LocalSystem (Windows service, auto start)	Session 0	Always	Start agent in the user session, restart it, verify signatures, apply updates
Desktop App (Electron)	Logged-in user	User session	Auto start at logon, tray	UI, network, commands the agent
Browser extension	Browser	User session	With browser	Report active tab domain

Why not a service alone: Windows services run isolated in Session 0. Calls such as `GetForegroundWindow` and `GetLastInputInfo` return nothing useful there. All measurement must happen inside the interactive user session.

The agent has no network code. It works identically with or without internet. All server communication is done by the Desktop App, which drains the agent queue when a connection exists (03 F7).

2. What the agent collects and how

Data	Windows mechanism	Notes
Foreground application	<code>SetWinEventHook(EVENT_SYSTEM_FOREGROUND)</code> + <code>GetForegroundWindow</code> , <code>GetWindowThreadProcessId</code> , process image path. 1 s polling as safety net	Process name, product name from file version info. UWP apps resolved through <code>ApplicationFrameHost</code> child window
Last input time	<code>GetLastInputInfo</code> every second	Basis for idle detection
Keyboard and mouse counts	Raw Input (<code>RegisterRawInputDevices</code> with <code>RIDEV_INPUTSINK</code>) on a hidden message window	Counter increments only. The key code field is never read, stored or logged. No low-level keyboard hook
Lock and unlock	<code>WTSRegisterSessionNotification</code> (<code>WTS_SESSION_LOCK</code> , <code>WTS_SESSION_UNLOCK</code>)	Lock starts a locked segment
Sleep, resume, shutdown	<code>PowerModeChanged</code> , <code>SessionEnding</code> events	Close segment, flush queue
Microphone in use	Core Audio API, active capture sessions	Activity exception mic
Website domain	Browser extension through native messaging	Section 5
Window title	<code>GetWindowText</code> , only if <code>capture_window_titles</code> is on	Off by default
Clock	<code>Stopwatch</code> (monotonic) anchored to server time given by the app	Wall clock changes detected and reported

Not collected, by design: key values, typed text, clipboard, screen content (1.0), audio, camera, files, full URLs.

3. Segments

A segment is one continuous period with the same kind, process and domain.

```
segment {
  id          UUID v7 (created by agent)
  shiftId
  startedAt, endedAt    UTC, from anchored monotonic clock
  kind          active | idle | locked
  processName   "chrome.exe"
  appName       "Google Chrome"
  domain        "youtube.com" | "unknown" | "excluded" | null
  windowTitle   null unless enabled
  keyCount, mouseCount
  exception     none | call_app | meeting_app | mic
  clockSource   anchored | unanchored
}
```

Segment is closed and a new one opened when: foreground process changes, domain changes, kind changes, pause or stop command, lock, power event, or the segment reaches 5 minutes.

Idle logic:

```
every second:
  sinceInput = now - lastInputTime
  if kind == active and sinceInput >= idleThreshold (default 300 s):
    if foreground process in exceptionApps or microphone in use:
      stay active, set exception
    else:
      split current segment at lastInputTime
      the part after lastInputTime becomes an idle segment
  if kind == idle and new input arrives:
    close idle segment, open active segment
    if idle duration >= longIdleThreshold: notify app (long idle prompt)
```

Switches shorter than 2 seconds (alt-tab flicker) are merged into the previous segment.

4. Local queue and pipe protocol

Queue: SQLite database in `%LOCALAPPDATA%\CompanyOS\agent\queue.db`, encrypted (SQLCipher or encrypted payload column) with a random key protected by DPAPI. Tables: `segments`, `state` (current shift, anchor, config), `tamper`.

- Segments are written when closed and additionally the open segment is checkpointed every 30 s, so a crash or power loss loses at most 30 s.
- Capacity [30] days or [500] MB. At 90 percent a warning is raised. Data is deleted only after the server confirmed it.
- After a PC restart with an open shift recorded in `state`, the agent resumes tracking as soon as the user logs in, without waiting for the app or the network.

Named pipe `\\.\pipe\CompanyOS.Agent.<sessionId>`, JSON lines, ACL for the current user only.

Command (app -> agent)	Meaning
hello {appVersion}	Handshake, returns agent version and state
setAnchor {serverTime}	Anchor the monotonic clock
setConfig {...}	Idle thresholds, exception apps, exclusions, capture flags
startTracking {shiftId}	Begin segments
pauseTracking {reason}	Break
resumeTracking	End of break
stopTracking	End shift, flush
pullSegments {max}	Returns queued segments
ackSegments {ids}	Delete confirmed segments
pullTamper / ackTamper	Tamper events
status	State, queue size, extension connected, last input

Event (agent -> app)	Meaning
longIdleEnded {from, to}	Show long idle prompt
queueWarning {percent}	Queue nearly full
stateChanged	Tracking state changed

5. Browser extension

Item	Specification
Type	Manifest V3, Chrome and Edge. Firefox in 1.1
Permissions	tabs, nativeMessaging. No content scripts, no page access, no history
Logic	On tabs.onActivated, tabs.onUpdated (URL change), windows.onFocusChanged: read the active tab URL, reduce to registrable domain (public suffix list), send {domain, browser, ts} to the native host
Native host	The agent registers a native messaging host manifest (com.companyos.agent) under HKLM for Chrome and Edge. A small stdio bridge forwards messages to the agent
Internal pages	chrome://, edge://, extension pages, local files -> domain browser-internal
Installation	Force-installed by policy ExtensionInstallForcelist (registry under HKLM written by the installer, or pushed from Google Admin console Chrome management). Users cannot remove a force-installed extension
Hosting	Chrome Web Store as unlisted item, or self-hosted update URL referenced in the policy
Incognito	Extensions do not run in incognito by default. Recommended policy: IncognitoModeAvailability = 1 (disabled). Otherwise domain is unknown
Other browsers	Time is recorded as application time without domain. Report shows them as "browser without extension". The client may restrict browsers by policy

Agent side: the domain reported by the extension is applied only while the foreground process is that browser. If no message arrived from the extension for 60 s while a supported browser is in the foreground, domain = `unknown` and a tamper event `extension_missing` is recorded (once per shift).

6. Watchdog service

1. Starts at boot. On user login (`WTS_SESSION_LOGON`) launches the agent in that session with the user's token (`WTSQueryUserToken` + `CreateProcessAsUser`).

2. Checks every 10 s that the agent is running, restarts it if not. Records `agent_stopped` with the gap length. Also relaunches the Desktop App (tray) if it is not running while a shift is open.
3. Verifies the Authenticode signature and publisher of the agent and app binaries before launch.
4. Performs the file replacement part of updates (the app downloads and verifies, the service installs), so no administrator prompt is shown to the employee.
5. Holds no credentials and has no network access.

Multiple Windows users on one PC: one agent per session, queue per user profile. Remote Desktop sessions are supported the same way.

7. Installer

- electron-builder NSIS, per-machine install to `Program Files`, requires elevation once (IT or admin installs).
- Installs: Desktop App, agent, watchdog service (registered and started), native messaging manifests, extension policy keys, autostart entry for the app (tray, minimised).
- Silent install switch for mass rollout (`/s`), suitable for deployment through endpoint management.
- Uninstall requires administrator rights.
- Everything signed. Submit the signed installer to the antivirus vendor used by the client for allowlisting before rollout.

8. Performance and quality budgets

Metric	Budget
Agent CPU	Under 1 percent average, under 3 percent peak
Agent memory	Under 80 MB
Disk writes	Batched, at most one write per 5 s outside segment closes
Data lost on power failure	At most 30 s
Segment timing accuracy	Within 2 s

9. Test matrix

Case	Expected
Switch between 5 apps quickly	Correct segments, flicker merged
No input 6 min in Word	Idle segment backdated to last input
No input 10 min in listed call app	Active with exception <code>call_app</code>
Lock screen 20 min	Locked segment, long idle prompt on unlock
Sleep 1 h, wake	Segment closed at sleep, tracking continues, gap offline
Unplug network 3 h, work, reconnect	Full data appears in Admin Panel after sync, flagged late
Network off, restart PC, work, reconnect next day	Shift resumed locally, all data synced, attendance recomputed
Kill agent in Task Manager	Restarted within 10 s, tamper event

Kill app during shift	Agent keeps tracking, app relaunched to tray, shift resumes
Change Windows clock by 3 h	Durations unchanged, clock_changed recorded
Incognito window	Domain unknown
Two monitors, video playing on second, typing on first	Foreground app only is counted
Duplicate upload of same batch	No double counting
Windows 10 and 11, standard user, with common antivirus	Install and run without blocks

10 - Google Workspace Email Integration

1. Principle

Employees never receive a Google password, token or session. The backend is the only Gmail API client. The Desktop App talks only to the backend, and the backend decides per request whether the employee may perform the action on that mailbox.

```
Desktop App --(our API, our permissions)--> Backend --(Gmail API)--> Google
```

Gmail API scopes are coarse (read, send, modify, full). Fine-grained rules such as "may reply but not delete" do not exist at Google. They are implemented as backend permission checks, and functions that must never happen (delete, settings, forwarding, filters) simply have no endpoint.

2. Connection method

Option	How	Pros	Cons	Decision
A. Per-mailbox OAuth, internal app	Admin clicks Connect and signs in once as each mailbox. Backend stores a refresh token per mailbox	Access exists only for connected mailboxes. Revocable per mailbox. No verification needed for an Internal app	One sign-in per mailbox	Chosen for 1.0
B. Service account with domain-wide delegation	Super Admin authorises the service account for the Gmail scope. Backend can act as any user	No per-mailbox sign-in, suits many mailboxes	The key can access every mailbox in the domain, including management. High impact if leaked	Alternative when more than about 20 mailboxes. Needs backend allowlist and strict key handling
C. IMAP and SMTP with app passwords	Store mailbox passwords	None	Stores passwords, weak, being phased out by Google	Rejected

A connectable mailbox must be a real Workspace user account (licensed mailbox). Aliases and Google Groups (collaborative inboxes) are not mailboxes and cannot be connected. Confirm with the client which addresses are user accounts (Q3).

With option A the OAuth client lives in a Google Cloud project owned by the client's Workspace organisation and has user type Internal. Internal apps are limited to the organisation's own users and do not go through Google's public verification. If the product is later sold to other companies (Q12), a public app with restricted Gmail scopes requires Google verification and a recurring security assessment. That is a separate project.

3. One-time setup (client Super Admin + developer)

1. Create a Google Cloud project inside the client's organisation.
2. Enable Gmail API and Cloud Pub/Sub API.
3. OAuth consent screen: user type Internal, scope `https://www.googleapis.com/auth/gmail.modify`.
4. Create OAuth client (Web application), redirect URI `https://api.<domain>/api/v1/mail/oauth/callback`.
5. Admin console, Security, API controls, App access control: mark the app as Trusted. If the organisation restricts Gmail to trusted apps (recommended), only this app can use the scope.
6. Create Pub/Sub topic `gmail-push`. Grant publisher role on the topic to `gmail-api-push@system.gserviceaccount.com`.
7. Create a push subscription to `https://api.<domain>/api/v1/mail/google/push` with OIDC authentication (dedicated service account, audience = the endpoint URL).
8. Store client id, client secret, topic name and expected OIDC service account in the backend secret manager.

Scope `gmail.modify` permits reading, sending, drafts, labels and moving to trash. It does not permit permanent deletion that bypasses Trash. The backend exposes no trash function at all.

4. Flows

4.1 Connect mailbox

```
Admin (mail.accounts.manage) -> Connect mailbox
-> redirect to Google (access_type=offline, prompt=consent, state=signed)
-> sign in as the mailbox (for example sales@company.com), consent
-> callback: exchange code, read profile (address must be in company domain)
-> store refresh token encrypted, token_status = ok
-> users.watch(topic, labelIds=[INBOX, SENT]) -> store historyId, expiry
-> queue initial sync (last 30 days of threads, metadata only)
-> audit entry
```

4.2 Incremental sync

```
Gmail change -> Pub/Sub push {emailAddress, historyId}
-> verify OIDC token -> enqueue gmail-sync(mailbox) (deduplicated)
-> users.history.list(startHistoryId = stored historyId)
-> for added/changed messages: messages.get(format=metadata)
-> upsert mail_threads / mail_messages, update unread, labels
-> store new historyId
-> WebSocket mail.new to assigned employees
-> link thread to lead when a participant address matches lead_emails
```

- History id too old (HTTP 404): run a full metadata resync for the last 30 days.
- Fallback: poll every 2 minutes per mailbox in case push is delayed.
- `gmail-watch-renew` runs daily. A watch expires after 7 days.
- Errors `invalid_grant`: set `token_status = revoked`, alert admins, show "Reconnect" in the Admin Panel, show "Mailbox unavailable" to employees.

4.3 Read

```
App: GET threads -> served from mail_threads (fast, searchable by subject,
participant). Full-text search passes q to Gmail (messages.list).
App: GET thread -> backend: permission read -> threads.get(format=full)
-> parse MIME -> sanitise HTML -> rewrite inline images (cid) to API URLs
-> response. Cached in Redis for 5 minutes, encrypted.
-> mail_audit(open)
```

Bodies are not stored in the database. This keeps mail content out of backups and limits what a database leak exposes.

4.4 Send, reply, forward, draft

```
App: POST send {to, cc, bcc, subject, html, attachments[], replyTo?}
-> permission: send (new, forward) or reply (reply); attach if files
-> validate recipients, size (25 MB total), attachment types
-> build MIME: From = mailbox, In-Reply-To + References + threadId on reply,
signature appended (mailbox signature + employee name if configured)
-> users.messages.send
-> mail_messages.sent_by_employee_id = employee
-> mail_audit(send | reply | forward)
-> matching lead: timeline entry, metric event emails_sent
```

Drafts are stored as Gmail drafts (`users.drafts`) so they are shared across employees assigned to the same mailbox and tagged with the author in our database.

5. Permission mapping

Assignment permission	Enables
read	Thread list, open thread, search
send	Compose new mail, forward
reply	Reply, reply all
draft	Create, edit, discard own drafts
attach	Add attachments when sending
download	Download received attachments
archive	Remove from Inbox (label change)
mark_read	Change read state

Never available to any role through this system: delete or trash, empty trash, account settings, forwarding addresses, filters, delegation, password, labels administration.

Recommended default for sales staff: read, send, reply, draft, attach, mark_read. `download` is granted deliberately because downloaded files leave the controlled environment.

6. Shared mailbox behaviour

- Several employees may be assigned to one mailbox. Read state is shared (it is the mailbox state).
- The thread view shows which employee sent each outgoing message (from `sent_by_employee_id`).
- Collision hint: when another assigned employee has the same thread open or a draft reply exists, the app shows "Ahmed is replying" (WebSocket presence, best effort).
- Optional thread assignment to one employee is a 1.1 item.

7. Limits and error handling

Topic	Handling
Gmail API quota	Per-mailbox token bucket in Redis, exponential backoff on 429 and 5xx, batch metadata requests
Sending limits	Google Workspace daily sending limits apply per mailbox. On limit error show a clear message and alert admins
Large mailboxes	Initial sync limited to 30 days, older mail reachable through search
Attachments	Streamed through the backend, never written to disk unencrypted, size cap 25 MB
Token secrecy	Refresh tokens encrypted (08). Access tokens kept in memory or Redis with TTL only
Disconnect	Revoke token at Google, delete stored token, keep audit and thread metadata per retention

8. Offboarding effect

Disabling an employee or removing an assignment needs no action at Google: the employee never held a credential. Access ends with the next API request. The mailbox, its password and its tokens are unaffected, so nothing has to be rotated when staff leave. This is the main security gain over sharing mailbox passwords.

9. Out of scope for 1.0

Calendar, contacts sync, label management, mail templates and sequences, tracking pixels, thread assignment, non-Google mail providers.

11 - Build Plan

Durations are planning estimates for a small team (1 to 2 developers) working with AI coding models. They are not commitments and should be re-estimated after Phase 1.

1. Phases

Phase	Name	Estimate	Depends on
P0	Foundation	1 week	-
P1	Identity, organisation, RBAC, devices, audit	2 weeks	P0
P2	Desktop shell, shift, attendance	2 weeks	P1
P3	Tracker agent, extension, activity reports	3 weeks	P2
P4	CRM, tasks, calls	3 weeks	P1
P5	Targets, notifications, dashboards	1.5 weeks	P3, P4
P6	Email	2.5 weeks	P1, P4
P7	Reports, hardening, installer, update, pilot	2 weeks	all
	Total	about 17 weeks	

P3 and P4 can run in parallel with two developers, which shortens the calendar time to about 13 weeks.

Risk-first note: build a throwaway spike of the agent (foreground app + idle + pipe + extension message) in the first week. It is the part with the most platform risk (antivirus, permissions, Session 0) and should be proven on a real employee PC early.

2. Phase details

P0 - Foundation

Deliverables: monorepo, lint and format, TypeScript strict, `packages/contracts`, NestJS skeleton with health endpoint, Prisma + first migration, Docker Compose (Postgres, Redis, MinIO), CI pipeline, staging environment, error tracking, seed script (company + Super Admin), `AGENTS.md`.

Acceptance: `pnpm dev` starts everything locally. CI green. Staging reachable over HTTPS.

P1 - Identity, organisation, RBAC, devices, audit

Deliverables: FR-AUTH-01 to 10, FR-DEV-01 to 05, FR-ORG-01 to 05, FR-RBAC-01 to 05, FR-AUD-01 to 03. Admin Panel: login with 2FA, employees, departments, teams, roles, devices, sessions, login history, audit viewer.

Acceptance:

- Flows F1, F3 (admin side), F4 pass end to end.
- Role matrix tests pass for all endpoints built so far.
- Disabled employee cannot log in. Revoked session is rejected within one request.
- Audit entries exist for every write.

P2 - Desktop shell, shift, attendance

Deliverables: Electron app with secure defaults, login, device registration and approval (F2), consent, token storage, WebSocket, tray, dashboard skeleton, shift control (F5 without agent data), heartbeat, offline queue for shift events, schedules, holidays, attendance computation (F8), live board, corrections and leave (F9), auto-close job.

Acceptance:

- F2, F3, F5, F8, F9 pass.
- Shift survives app restart. Start, break and end work with the network cable unplugged and sync afterwards.
- Night shift 20:00 to 04:00 is attributed to one attendance date.
- Token copied to another PC cannot refresh.

P3 - Tracker agent, extension, activity reports

Deliverables: agent, watchdog, pipe protocol, local encrypted queue, extension + native host, ingest endpoint, rollups, partitioning, productivity rules, exclusions, exception apps, tamper events, Admin Panel views (time tracking, activity drill-down, applications, websites), own activity view, long idle prompt.

Acceptance: the complete test matrix in 09 section 9 passes on Windows 10 and Windows 11 with a standard user account. In particular: 3 hours offline work and an offline PC restart both appear completely in the Admin Panel after reconnect. Drill-down matches the original example (app, domain, time ranges).

P4 - CRM, tasks, calls

Deliverables: FR-CRM-01 to 14, FR-TASK-01 to 04, pipelines config, list and detail in the Desktop App, admin CRM pages, import job, duplicate check, call logging, appointments, follow-ups, attachments, timeline.

Acceptance: F10, F11, F12 pass. Employee cannot see or guess another employee's lead (404). Import of 10,000 rows completes with a duplicate report. Export is blocked without permission and audited with it.

P5 - Targets, notifications, dashboards

Deliverables: metric events, targets, progress, snapshots, leaderboard, notifications store + WebSocket + toasts, employee dashboard complete, admin dashboard.

Acceptance: F13, F15 pass. Logging a connected call moves both call metrics once. Editing or resending the same call does not double count. Period rollover creates a snapshot.

P6 - Email

Deliverables: Google setup with the client, connect flow, sync worker, watch renewal, thread list and view, sanitiser, compose, reply, forward, drafts, attachments, assignments with permissions, mail audit, lead linking, mailbox health.

Acceptance: F14 passes. An employee without `send` cannot send through any endpoint. No endpoint can delete mail. New mail appears in the app within 15 seconds. Revoking the token

at Google produces an admin alert and a reconnect action. Unassigning removes access immediately.

P7 - Reports, hardening, installer, update, pilot

Deliverables: all reports with CSV export, retention purge, backups and restore test, rate limits, security checklist (08 section 4), signed installer, auto-update with channels, mandatory update, silent install, load test, runbook, admin and employee user guides, pilot with 5 to 10 employees for 2 weeks, fixes.

Acceptance: F16, F17, F18, F19 pass. Load test with 200 simulated agents. Restore from backup proven. Pilot exit criteria: no data loss, tracker accuracy within 2 percent of a manual sample, no antivirus blocks, sign-off by the client.

3. Definition of done (every feature)

- 1. Matches the referenced FR ids and flows.
- 2. Zod contract in `packages/contracts`, used by server and client.
- 3. Permission and scope enforced on the server, with tests per role.
- 4. Audit entry for every write.
- 5. Unit tests for business rules, integration test for the endpoint.
- 6. Loading, empty and error states in the UI.
- 7. No secrets in code, no new lint or type errors.
- 8. Migration included and reversible in staging.
- 9. Documentation updated when behaviour deviates from these files.

4. Testing strategy

Level	Tool	Coverage
Unit	Vitest (TS), xUnit (.NET)	Rules: shift state machine, attendance status, idle splitting, target periods, scope resolution
Integration	Vitest + Testcontainers (Postgres, Redis)	Every endpoint, role matrix, idempotency
Contract	Generated OpenAPI + schema tests	Client and server stay in sync
End to end, admin	Playwright	Main admin flows
End to end, desktop	Playwright for Electron	Login, shift, CRM, mail basics
Agent	xUnit + scripted manual matrix on real PCs	09 section 9
Load	k6	Ingest, heartbeat, reports
Security	Dependency scan, ASVS checklist, external penetration test (recommended)	Before rollout

Time-dependent logic (attendance, targets, auto-close) MUST use an injectable clock so tests can simulate days and timezones.

5. Rollout

- 1. Staging with test accounts.

2. Pilot channel: 5 to 10 employees, one team, 2 weeks. Daily check of tracker data against reality.
3. Monitoring policy communicated to all staff by the client before company-wide rollout.
4. Rollout by department. Installer deployed silently by IT.
5. Stable channel updates follow pilot by at least 3 days.

6. Risks

Risk	Impact	Mitigation
Antivirus flags the agent	Rollout blocked	Code signing from the first pilot build, no keyboard hooks, vendor allowlisting, early spike
Employees have local admin rights	Tracking can be defeated	Standard user accounts through endpoint management, tamper alerts
Google setup delayed by client	Email phase blocked	Request the setup in P1, email is last feature phase
Scope growth (dialer, screenshots, payroll)	Delay	Open questions decided before P4, change requests priced separately
Legal objection to monitoring	Feature removal	Transparency design, policy, client legal confirmation before pilot
AI-generated code skips scope checks	Data leak between employees	Mandatory role-matrix tests, review checklist, guard + repository pattern
Clock and timezone bugs	Wrong attendance	Server time, injectable clock, night-shift tests
Large activity tables	Slow reports	Partitioning, rollups, retention from day one

7. Items needed from the client

Answers to Q1 to Q12 (01), Google Workspace Super Admin availability for the setup in 10, code signing certificate, hosting account, list of mailboxes and assignments, shift schedules and holidays, employee list with roles, monitoring policy text approved by their adviser, pilot team.

12 - AI Build Guide

How to build this system with AI coding models without losing consistency.

1. Working method

1. Put all files of this documentation into `docs/` in the repository. Put the rules from section 3 into `AGENTS.md` (and `CLAUDE.md` if the tool reads that name) at the repository root.
2. Work in small vertical slices: one feature = contract + migration + endpoint + tests + UI. Never ask for a whole module in one prompt.
3. For each slice give the model: `AGENTS.md`, the relevant flow from 03, the FR ids from 04, the tables from 06, the endpoints from 07, and the existing code of the neighbouring feature as a pattern.
4. Build the first feature of each kind by hand-reviewing closely (one CRUD endpoint, one guarded list, one job, one admin page, one desktop page). These become the reference patterns the model copies.
5. Contracts first. Change `packages/contracts`, then server, then clients. Never let the model invent a field in the UI that the contract lacks.
6. Tests are part of the prompt, not a later step. Require the role-matrix test for every endpoint.
7. Run lint, typecheck and tests after every change. Paste failures back to the model instead of fixing around them.
8. One branch and one pull request per slice. Review the diff yourself. Pay most attention to: scope filters, `company_id` handling, migrations, anything touching tokens or mail.
9. When the model proposes something that contradicts the docs, either reject it or update the docs first. The docs stay the source of truth.

2. Suggested build order (slices)

```
P0 repo, contracts package, api skeleton, db, ci
P1 auth login -> refresh -> device registration -> admin 2fa
  -> employees crud -> roles/permissions + guard -> audit interceptor
  -> devices admin -> sessions admin
P2 electron shell (secure) -> login + device pending -> consent
  -> shift events api -> shift ui + offline queue -> heartbeat + auto-close
  -> schedules -> attendance job -> live board -> corrections
P3 agent spike -> pipe protocol -> segments + queue -> ingest api
  -> rollups -> extension + native host -> watchdog -> admin activity views
P4 pipelines -> leads crud + scope -> timeline -> notes -> calls
  -> appointments + follow-ups -> tasks -> import -> export
P5 metric events -> targets -> progress ws -> notifications -> dashboards
P6 oauth connect -> sync worker -> thread list -> thread view + sanitiser
  -> send/reply -> drafts -> assignments -> audit -> lead linking
P7 reports -> retention -> installer + signing -> auto-update -> load test
```

3. AGENTS.md (copy into the repository root)

```

# Project rules for AI coding agents

## Source of truth
- docs/ is authoritative. Read the files named in the task before coding.
- If the task conflicts with docs/, stop and say so. Do not improvise.

## Stack
- Backend: NestJS, TypeScript strict, Prisma, PostgreSQL 16, Redis, BullMQ.
- Admin: Next.js App Router, React, Tailwind, shadcn/ui, TanStack Query.
- Desktop: Electron (electron-vite), React. Agent: C# .NET 8.
- Shared: packages/contracts (Zod). All DTOs, enums, permission keys,
  WebSocket events come from there.

## Hard rules
1. Every endpoint has @RequirePermission and uses the ScopeFilter in its
  repository query. No query on employee-owned data without it.
2. company_id comes from the session only. Never from the request body.
3. Every write creates an audit log entry through the audit interceptor.
4. Validate all input with the Zod schema from packages/contracts.
5. Timestamps: timestamptz in UTC. Use the injected Clock service, never
  Date.now() or new Date() in business logic.
6. No secrets, tokens, passwords or mail content in logs.
7. Desktop renderer has no Node access. All privileged work goes through
  the preload API. Tokens stay in the main process.
8. The tracker never records key values, clipboard, or screen content.
  Counts only. Do not add a keyboard hook.
9. The agent has no network code. Tracking must work with no internet.
10. Mail: no delete/trash/settings/filter endpoints. The backend is the
  only Gmail client.
11. Idempotency: desktop writes accept Idempotency-Key; ingest upserts by
  client UUID.
12. Migrations are forward-only and reviewed. Never edit an applied one.
13. Do not add dependencies without stating why. Pin versions.
14. Lists are paginated (cursor). No unbounded queries.

## Every feature must include
- contract (Zod) -> migration -> service -> controller -> tests -> UI
- tests: unit for rules, integration for the endpoint, one allowed and
  one denied case per role
- loading, empty and error states in the UI

## Style
- Small modules, one responsibility. No dead code. No TODO left behind
  without an issue reference.
- Error codes from packages/contracts/errors.ts.
- Commit message: "<area>: <change> (FR-XXX-NN)".

## When unsure
Ask. List the options and the doc section that is ambiguous.

```

4. Prompt template for one slice

Task: Implement <feature name>.

Read first:

- AGENTS.md
- docs/03-flows.md section <F..>
- docs/04-modules-spec.md requirements <FR-...>
- docs/06-data-model.md tables <...>
- docs/07-api-spec.md endpoints <...>
- Reference implementation to follow: <path of similar feature>

Deliver, in this order:

1. Zod schemas in packages/contracts
2. Prisma migration
3. Service + controller with permission and scope
4. Tests (unit, integration, role matrix)
5. UI: <admin page | desktop page> with loading/empty/error states

Constraints:

- Do not change unrelated files.
- Do not add libraries.
- Stop and ask if the docs do not cover a case.

Finish with: list of files changed, how to test manually, open questions.

5. Review checklist for AI output

Check	Question
Scope	Does every query filter by company and by the caller's scope?
Identity	Is the acting employee taken from the session, never from input?
Audit	Is there an audit entry for each write?
Contract	Do client and server use the same schema from contracts?
Time	Is the Clock service used? Are periods computed in company timezone?
Idempotency	Can the request be repeated without duplicates?
Errors	Are error codes from the shared list, and no internals leaked?
Tests	Is there a denied-access test, not only a happy path?
Secrets	Anything sensitive logged or returned?
Offline	Desktop feature: what happens with no network?
Docs	Does behaviour match the flow? If not, which one changes?

6. Known traps for AI-generated code in this project

1. Tracker written as a Windows service only. It will compile and record nothing useful. Measurement belongs in the user-session agent.
2. Keyboard counting through a low-level hook that reads key codes. Use Raw Input counters.
3. Using the PC clock for attendance. Server time and monotonic durations only.
4. Giving the Electron renderer direct access to tokens or Node.
5. Storing Gmail tokens unencrypted or sending Google tokens to the desktop.
6. Scope check in the controller but an unfiltered query in the service.
7. Midnight-based "today" for night shifts. Use the attendance date.
8. Double counting metrics on retries. Use the unique key on metric_events.
9. Rendering email HTML directly in the app DOM.

10. Stopping the tracker when an upload fails. Upload and tracking are independent.